

ĐẠI HỌC QUỐC GIA HÀ NỘI
TRƯỜNG ĐẠI HỌC KHOA HỌC TỰ NHIÊN



BÁO CÁO TIỂU LUẬN

HỌC PHẦN: Cơ sở dữ liệu
CHỦ ĐỀ: HADOOP

Mã học phần : MAT3507
Giảng viên : TS. Vũ Tiến Dũng

Họ và tên	Lớp	Mã SV
Trương Thị Yến Nhi	K68A2 Toán Tin	23000148
Nguyễn Văn Hanh	K68A2 Toán Tin	23000117
Đỗ Minh Hoàng	K68A2 Toán Tin	23000121
Trần Thị Quỳnh Trang	K68A2 Toán Tin	23000163

Hà Nội - 2026

MỤC LỤC

CHỦ ĐỀ: TÌM HIỂU VỀ HADOOP.....	0
Phần 1: Big Data.....	4
1.1 Định nghĩa.....	4
1.2 Đặc trưng.....	4
1.2.1. Volume - Khối lượng dữ liệu.....	4
1.2.2. Variety – Tính đa dạng.....	4
1.2.3. Velocity – Tốc độ xử lý.....	5
1.2.4. Veracity – Tính xác thực.....	5
1.2.5. Value – Giá trị.....	5
1.3. Phân loại.....	5
1.3.1. Dữ liệu có cấu trúc (Structured data).....	5
1.3.2. Dữ liệu bán cấu trúc (Semi structured data).....	5
1.3.3. Dữ liệu phi cấu trúc (Unstructured data).....	6
1.4 .Nguyên tắc hoạt động của Big Data Analytics là gì?.....	6
1.4.1. Thu thập.....	7
1.4.2. Lưu trữ.....	7
1.4.3. Xử lý và phân tích.....	7
1.4.4. Trực quan hóa và sử dụng.....	6
1.5 .Ứng dụng của Big Data.....	6
1.5.1 Lợi ích khi làm chủ được Big Data.....	6
1.5.2 Ứng dụng của Big Data trong lĩnh vực kinh tế.....	7
1.5.3 Ứng dụng của Big Data trong lĩnh vực tài chính ngân hàng.....	9
1.6 Cơ hội và thách thức khi sử dụng Big Data.....	9
1.6.1 Cơ hội.....	9
1.6.2 Thách thức.....	10
Phần 2 . NoSQL.....	11
2.1. Định nghĩa.....	11
2.2. Ưu - nhược điểm của cơ sở dữ liệu NoSQL.....	11
2.2.1. Ưu điểm.....	11
2.2.2. Nhược điểm.....	12
2.3. Phân loại cơ sở dữ liệu NoSQL.....	13
2.3.1. Cơ sở dữ liệu Key-Value.....	13
2.3.2. Cơ sở dữ liệu Document.....	14
2.3.3. Cơ sở dữ liệu Column-Family.....	16
2.3.4. Cơ sở dữ liệu Graph.....	16
2.4. So sánh NoSQL với SQL.....	18
2.5. Kết luận.....	20
Phần 3 : HADOOP.....	21
3.1 Hadoop là gì?.....	21
3.2. Lịch sử phát triển và các phiên bản của Hadoop.....	21
3.3. Kiến trúc tổng quan của Hadoop.....	22
3.3.1. Hadoop Distributed File System (HDFS).....	23
3.3.2. Hadoop MapReduce.....	23

3.3.3. Hadoop Common.....	24
3.3.4. Hadoop YARN.....	24
Phần 4: YARN (Yet Another Resource Negotiator).....	24
4.1 Kiến trúc và cơ chế hoạt động của Yarn.....	24
4.2 Cơ chế lập lịch tài nguyên trong Yarn (FIFO, Capacity Scheduler, Fair Scheduler).....	25
4.3 Các cải tiến của Yarn trong Hadoop 3.x.....	26
4.4 Ưu điểm và nhược điểm của YARN.....	29
4.4.1 Ưu điểm của YARN.....	29
4.4.2 Nhược điểm của YARN.....	30
Phần 5 . Mô hình lập trình phân tán MapReduce.....	32
5.1 Giới thiệu về MapReduce.....	32
5.2 Kiến trúc và cách thức hoạt động của MapReduce.....	32
5.3 Ưu điểm và nhược điểm của MapReduce.....	32
5.3.1 Ưu điểm.....	32
5.3.2 Nhược điểm.....	33
Phần 6. HDFS – Hệ thống lưu trữ phân tán trong Hadoop.....	35
6.1 Giới thiệu về HDFS (Hadoop Distributed File System).....	35
6.2 Cấu trúc và nguyên tắc hoạt động của HDFS.....	35
6.2.1 Cấu trúc của HDFS.....	35
6.2.2 Nguyên tắc hoạt động của HDFS.....	36
6.3 Ưu điểm và nhược điểm của HDFS.....	36
6.3.1 Ưu điểm của HDFS.....	36
6.3.2 Nhược điểm của HDFS.....	36
Phần 7. Một số công cụ và hệ sinh thái mở rộng của Hadoop.....	39
7.1. Apache Hive – Hệ thống kho dữ liệu trên Hadoop.....	39
7.2. Apache Pig – Ngôn ngữ lập trình dữ liệu lớn trên Hadoop.....	40
7.3. Apache HBase – Cơ sở dữ liệu NoSQL trên Hadoop.....	41
Phần 8. Thách thức và xu hướng phát triển của Hadoop và Big Data.....	42
8.1. Những thách thức khi triển khai Hadoop.....	42
8.2. Các phương pháp tối ưu hóa Hadoop.....	42
8.3. Hadoop có đang dần bị thay thế bởi các công nghệ mới không?...43	43
8.4. Những cải tiến trong các phiên bản mới của Hadoop.....	43
8.5. Vai trò của Hadoop trong chiến lược dữ liệu của doanh nghiệp hiện đại.....	44
Phần 9. Kết luận.....	45
9.1. Tổng quan về Hadoop, YARN, MapReduce và Hệ sinh thái Big Data.....	45
9.2. Trường hợp ứng dụng Hadoop.....	45
9.3. Xu hướng phát triển của Hadoop và Big Data trong bối cảnh AI và Điện toán đám mây (Cloud).....	46
Tài liệu tham khảo.....	47

Mục lục hình ảnh

Hình 1 . 5V của Big Data.....	4
Hình 2 . Ví dụ JSON4.....	6
Hình 3 . Quy trình 4 bước của Big Data.....	7
Hình 4 . Ứng dụng của Big Data trong y tế.....	9
Hình 5 . Các cơ sở dữ liệu NoSQL.....	11
Hình 6 . Ưu điểm của cơ sở dữ liệu NoSQL.....	12
Hình 7 . Cơ sở dữ liệu Key-Value.....	13
Hình 8 . Cơ sở dữ liệu Document.....	14
Hình 9 . Cơ sở dữ liệu Column-Family.....	16
Hình 10 Cơ sở dữ liệu Graph.....	17
Hình 11 . Hình ảnh về phân loại các CSDL NoSQL.....	18
Hình 12. Tổng quan về Hadoop.....	22
Hình 13 . Kiến trúc của YARN.....	24
Hình 14 . Cấu trúc của HDFS.....	35

Phần 1: Big Data

1.1 Định nghĩa

Dữ liệu lớn (Big data) là một thuật ngữ chỉ bộ dữ liệu lớn hoặc phức tạp mà các phương pháp truyền thống không đủ các ứng dụng để xử lý dữ liệu này. Thường bao gồm tập hợp dữ liệu với kích thước vượt xa khả năng của các công cụ phần mềm thông thường để thu thập, hiển thị, quản lý và xử lý dữ liệu trong một thời gian có thể chấp nhận được.

Big data có thể là dữ liệu có cấu trúc, bán cấu trúc hoặc không có cấu trúc

1.2 Đặc trưng

Dữ liệu lớn có 5 đặc trưng cơ bản được gọi là "5Vs": volume (khối lượng), velocity (tốc độ), variety (đa dạng), veracity (độ tin cậy), value (giá trị)



Hình 1. 5V của Big Data

1.2.1 Volume - Khối lượng dữ liệu

Đây là đặc điểm tiêu biểu nhất của dữ liệu lớn, khối lượng dữ liệu rất lớn. Kích cỡ của Big Data đang từng ngày tăng lên, và tính đến năm 2012 thì nó có thể nằm trong khoảng vài chục terabyte cho đến nhiều petabyte (1 petabyte = 1024 terabyte) chỉ cho một tập hợp dữ liệu. Dữ liệu truyền thống chúng ta có thể lưu trữ trên các thiết bị đĩa mềm, đĩa cứng. Nhưng với dữ liệu lớn sẽ sử dụng công nghệ “đám mây” mới có khả năng lưu trữ được dữ liệu lớn.

1.2.2 Variety – Tính đa dạng

Đối với dữ liệu truyền thống chúng ta hay nói đến dữ liệu có cấu trúc, thì ngày nay hơn 80% dữ liệu được sinh ra là phi cấu trúc (tài liệu, blog, hình ảnh, vi đeo, bài hát, dữ liệu từ thiết bị cảm biến vật lý, thiết bị chăm sóc sức khỏe...). Big Data cho phép liên kết và phân tích nhiều

dạng dữ liệu khác nhau. Ví dụ, với các comments/post của một nhóm người dùng nào đó trên Facebook với thông tin video được chia sẻ từ Youtube và Twitter.

1.2.3 Velocity – Tốc độ xử lý

Tốc độ có thể hiểu theo 2 khía cạnh: (a) Khối lượng dữ liệu gia tăng rất nhanh (mỗi giây có tới 72.9 triệu các yêu cầu truy cập tìm kiếm trên web bán hàng của Amazon); (b) Xử lý dữ liệu nhanh ở mức thời gian thực (real-time), có nghĩa dữ liệu được xử lý ngay tức thời ngay sau khi chúng phát sinh (tính đến bằng mili giây). Các ứng dụng phổ biến trên lĩnh vực Internet, Tài chính, Ngân hàng, Hàng không, Quân sự, Y tế – Sức khỏe như hiện nay phần lớn dữ liệu lớn được xử lý real-time. Công nghệ xử lý dữ liệu lớn ngày một tiên tiến cho phép chúng ta xử lý tức thì trước khi chúng được lưu trữ vào cơ sở dữ liệu.

1.2.4 Veracity – Tính xác thực

Một trong những tính chất phức tạp nhất của BigData là độ tin cậy/chính xác của dữ liệu. Với xu hướng phương tiện truyền thông xã hội (Social Media) và mạng xã hội (Social Network) ngày nay và sự gia tăng mạnh mẽ tính tương tác và chia sẻ của người dùng Mobile làm cho bức tranh xác định về độ tin cậy và chính xác của dữ liệu ngày một khó khăn hơn. Bài toán phân tích và loại bỏ dữ liệu thiếu chính xác và nhiễu đang là tính chất quan trọng của BigData.

1.2.5 Value – Giá trị

Giá trị là đặc điểm quan trọng nhất của dữ liệu lớn, vì khi bắt đầu triển khai xây dựng dữ liệu lớn thì việc đầu tiên chúng ta cần phải làm đó là xác định được giá trị của thông tin mang lại như thế nào, khi đó chúng ta mới có quyết định nên triển khai dữ liệu lớn hay không. Nếu chúng ta có dữ liệu lớn mà chỉ nhận được 1% lợi ích từ nó, thì không nên đầu tư dữ liệu lớn. Kết quả dự báo chính xác thể hiện rõ nét nhất về giá trị của dữ liệu lớn mang lại. Ví dụ, từ khối dữ liệu phát sinh trong quá trình khám, chữa bệnh sẽ giúp dự báo về sức khỏe được chính xác hơn, sẽ giảm được chi phí điều trị và các chi phí liên quan đến y tế.

1.3 Phân loại

Có 3 kiểu dữ liệu phổ biến: dữ liệu có cấu trúc, dữ liệu phi cấu trúc và dữ liệu bán cấu trúc

1.3.1 Dữ liệu có cấu trúc (Structured data)

Dữ liệu có cấu trúc là dữ liệu có định dạng chuẩn hóa để con người cũng như phần mềm có thể truy cập một cách hiệu quả. Loại dữ liệu này thường ở dạng bảng, bao gồm các hàng và cột xác định rõ ràng các thuộc tính dữ liệu. Với tính chất định lượng, dữ liệu có cấu trúc có thể được máy tính xử lý hiệu quả để thu thập thông tin chi tiết.

Ví dụ: bảng dữ liệu có cấu trúc về khách hàng bao gồm các cột tên, địa chỉ và số điện thoại có thể cung cấp các thông tin chi tiết như tổng số khách hàng và khu vực có nhiều khách hàng nhất. Ngược lại, dữ liệu phi cấu trúc, chẳng hạn như danh sách các bài đăng trên mạng xã hội, sẽ khó phân tích hơn.

1.3.2 Dữ liệu bán cấu trúc (Semi structured data)

Dữ liệu bán cấu trúc là dạng lai giữa dữ liệu có cấu trúc và dữ liệu phi cấu trúc. Nó không được tổ chức rõ ràng như dữ liệu có cấu trúc nhưng vẫn có mức tổ chức nhất định. Lấy JSON, một định dạng dữ liệu phổ biến. Trong định dạng này, các khóa như " **name** ", " **age** " và " **email** " cung cấp mức độ tổ chức cơ bản bằng cách dán nhãn rõ ràng cho từng phần tử dữ liệu. Điều này giúp dễ hiểu và truy cập các thông tin cụ thể. Ví dụ, một bản ghi có thể trông như thế này:

```
{  
  "name": "John Doe",  
  "age": 30,  
  "email": "john.doe@example.com"  
}
```

Hình 2. Ví dụ JSON

Dữ liệu bán cấu trúc là thông tin không nằm trong cơ sở dữ liệu quan hệ nhưng có một số thuộc tính tổ chức giúp phân tích dễ dàng hơn.

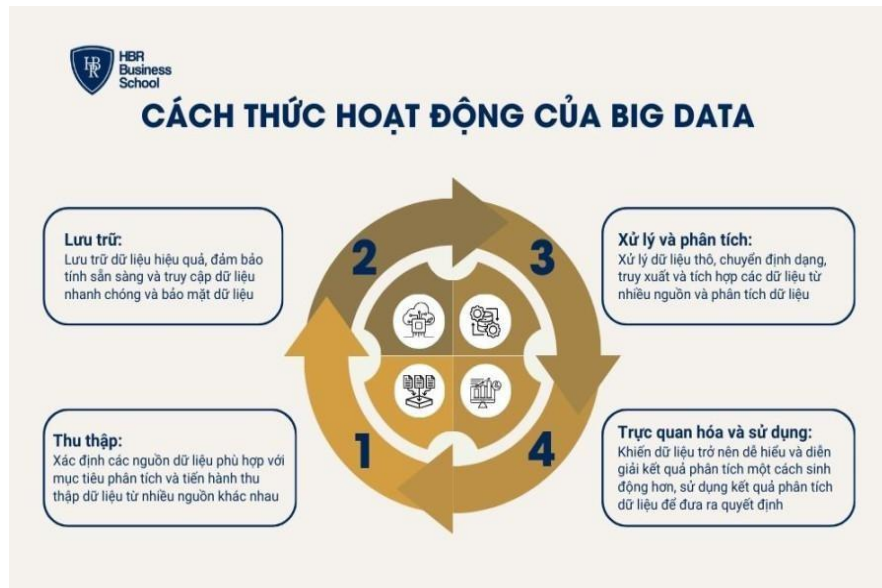
1.3.3 Dữ liệu phi cấu trúc (Unstructured data)

Dữ liệu phi cấu trúc là loại dữ liệu không có cấu trúc xác định, không được lưu trữ dưới dạng bảng (như SQL) hoặc định dạng có quy tắc (như JSON/XML). Dữ liệu này chiếm hơn 80% tổng dữ liệu hiện nay và đòi hỏi các công nghệ đặc biệt để xử lý.

1.4 .Nguyên tắc hoạt động của Big Data Analytics là gì?

Nguyên tắc hoạt động cơ bản của quá trình phân tích dữ liệu lớn bao gồm các bước: thu thập dữ liệu, xử lý dữ liệu, phân tích dữ liệu, trực quan hóa dữ liệu

Để thực hiện các quy trình phân tích này, doanh nghiệp có thể áp dụng nhiều công nghệ và phương pháp khác nhau, bao gồm các kỹ thuật thống kê, khai phá dữ liệu, học máy, và trí tuệ nhân tạo



Hình 3. Quy trình 4 bước hoạt động của Big Data

1.4.1 Thu thập

Ở bước này, doanh nghiệp cần xác định các nguồn dữ liệu phù hợp với mục tiêu phân tích và tiến hành thu thập dữ liệu từ nhiều nguồn khác nhau, bao gồm dữ liệu nội bộ, dữ liệu bên ngoài và dữ liệu theo thời gian thực. Để tối ưu quá trình thu thập, doanh nghiệp nên sử dụng các công cụ và kỹ thuật để đảm bảo tính chính xác và đầy đủ của dữ liệu.

Ví dụ: Walmart, nhà bán lẻ lớn nhất thế giới đã ứng dụng Big Data để cải thiện trải nghiệm khách hàng và thúc đẩy doanh số bán hàng. Thương hiệu này đã tiến hành thu thập dữ liệu từ nhiều nguồn khác nhau, bao gồm: dữ liệu bán hàng, dữ liệu khách hàng, dữ liệu mạng xã hội...

1.4.2 Lưu trữ

Sau khi thu thập, dữ liệu cần được lưu trữ dữ liệu một cách an toàn và hiệu quả. Hiện nay các doanh nghiệp sử dụng các hệ thống lưu trữ Big Data chuyên dụng như Hadoop, HDFS, Cassandra,... Dữ liệu lưu trữ cần đảm bảo tính sẵn sàng và truy cập dữ liệu nhanh chóng để đáp ứng nhu cầu phân tích. Nhằm bảo vệ dữ liệu khỏi các mối đe dọa an ninh mạng, doanh nghiệp nên lưu ý triển khai các biện pháp bảo mật.

Ví dụ: Thương hiệu Walmart đã sử dụng hệ thống lưu trữ dữ liệu Big Data Hadoop để lưu trữ lượng dữ liệu khổng lồ một cách an toàn và hiệu quả. Hệ thống này cho phép Walmart truy cập và phân tích dữ liệu nhanh chóng để đáp ứng nhu cầu kinh doanh và tiếp thị.

1.4.3 Xử lý và phân tích

Bao gồm các bước như:

- Xử lý dữ liệu thô, loại bỏ các lỗi, thiếu sót và dữ liệu trùng lặp
- Chuyển đổi định dạng dữ liệu để phù hợp cho việc phân tích.
- Truy xuất và tích hợp các dữ liệu từ nhiều nguồn khác nhau để tạo ra một tập dữ liệu thống nhất.
- Phân tích dữ liệu bằng các công cụ và kỹ thuật Big Data như Apache Spark,

MapReduce, Machine Learning, SQL, Python...

Ví dụ: Walmart sử dụng các công cụ và kỹ thuật Big Data như Apache Spark, MapReduce, Machine Learning để phân tích dữ liệu thu thập được.

1.4.4 Trực quan hóa và sử dụng

Sau khi phân tích, dữ liệu nên được trực quan hóa. Vậy mục đích trực quan hoá dữ liệu Big Data là gì? Đó là để dữ liệu trở nên dễ hiểu và diễn giải kết quả phân tích một cách sinh động hơn. Một số công cụ trực quan hoá dữ liệu là như Power BI, Tableau, Google Data Studio... Doanh nghiệp sẽ sử dụng kết quả phân tích dữ liệu để đưa ra quyết định sáng suốt trong kinh doanh. Từ đó, doanh nghiệp cải thiện được hiệu quả hoạt động kinh doanh, nâng cao vị thế cạnh tranh và thúc đẩy tăng trưởng doanh nghiệp.

Ví dụ: Walmart sử dụng kết quả phân tích dữ liệu để đưa ra quyết định trong kinh doanh, bao gồm Cá nhân hóa trải nghiệm mua sắm cho từng khách hàng; Tối ưu hóa quản lý hàng tồn kho và chuỗi cung ứng; Triển khai các chương trình khuyến mãi và tiếp thị hiệu quả.

1.5 Ứng dụng của Big Data

1.5.1 Lợi ích khi làm chủ được Big Data

Làm chủ được big data thể hiện ở các mặt như: lưu trữ được, phân tích được, thu thập được. Trong thời buổi hiện nay, data được xem như một dạng tài nguyên quý báu không kém gì vàng, bạc cả. Ví dụ giữa việc bạn nắm trong tay một danh sách thông tin người dùng của trang mạng xã hội facebook, so với việc bạn vô tình nhặt được một viên kim cương. Thì cả hai trường hợp này đều có khả năng giúp bạn kiếm được tiền – tức đem lại lợi nhuận. Đặc biệt hơn, big data còn đóng vai trò rất lớn trong việc đưa các chỉ số. Ví dụ nắm trong tay tất cả các bức ảnh mà chị em phụ nữ check in trên mạng xã hội, mình có thể phân tích và đưa ra xu hướng thời trang hiện tại, cũng như có thể dự đoán xu hướng sắp tới. Nhờ đó mà mình biết rằng nên nhập mẫu thời trang nào về bán sẽ nhận được nhiều sự quan tâm nhất. Big data cũng có vai trò lớn trong việc phát triển AI (trí tuệ nhân tạo), bởi một trong số những yếu tố quyết định AI có thông minh hay không chính là nằm ở tập dữ liệu mà chúng ta cho nó học. Làm chủ được Big data sẽ gia tăng cơ hội kiếm tiền.

1.5.2 Ứng dụng của Big Data trong lĩnh vực y tế

Trong y học các bác sĩ dựa vào số liệu trong các bệnh án để đưa ra dự đoán về nguy cơ mắc bệnh. Đồng thời cũng đưa ra được xu hướng lây lan của bệnh. Dữ liệu của bệnh nhân được lưu trữ trong cơ sở dữ liệu an toàn và bảo mật. Các chuyên gia y tế có thể theo dõi lịch sử của bệnh nhân, liên kết với những ứng dụng theo dõi sức khỏe mà bệnh nhân và người điều trị của họ có quyền truy cập.

Ứng dụng của big data trong y tế gồm:

- Dự đoán bệnh nhân để cải thiện nhân sự
- Hồ sơ sức khỏe điện tử
- Cảnh báo thời gian thực

HADOOP

- Tăng cường sự tham gia của bệnh nhân

Chăm sóc sức khỏe là lĩnh vực đòi hỏi sự chính xác cao cũng như một hệ thống thông tin xuyên suốt cho các bệnh nhân nói riêng hay cả cộng đồng dân cư nói chung. Với khối lượng thông tin khổng lồ cùng hàng nghìn dữ liệu cần được phân tích, xử lý, ứng dụng của Big Data chính là giải pháp cốt lõi giúp nâng cao chất lượng của các dịch vụ y tế trên toàn cầu.



Hình 4. Ứng dụng của Big Data trong y tế

1.5.3 Ngành tài chính ngân hàng

Ứng dụng Big Data trong ngành Tài chính - Ngân hàng tại Việt Nam sẽ giúp các doanh nghiệp có thể:

- Phân tích các thói quen chi tiêu của khách hàng
- Phân khúc khách hàng và thẩm định hồ sơ
- Bán chéo thêm các dịch vụ khác
- Nâng cao chất lượng dịch vụ thông qua xây dựng hệ thống thu thập các phản hồi từ khách hàng và phân tích chúng
- Marketing theo hướng cá nhân hóa
- Thay đổi cách thức cung cấp dịch vụ đến khách hàng
- Phát hiện và ngăn chặn hành vi lừa đảo, vi phạm pháp luật
- Kiểm soát rủi ro, tuân thủ luật pháp và minh bạch

Tiềm năng của phân tích dữ liệu lớn (Big Data) đã được khẳng định rõ ràng trong ngành tài chính toàn cầu, với doanh thu từ lĩnh vực này tăng từ 130 tỷ USD (2016) lên 203 tỷ USD (2020), trong đó ngân hàng là lĩnh vực đóng góp lớn nhất. Big Data giúp ngân hàng quản trị rủi ro hiệu quả hơn (từ rủi ro tín dụng, thị trường đến rủi ro tác nghiệp), đồng thời nâng cao chất lượng dịch vụ, dự báo kinh doanh và lập kế hoạch chiến lược. Đối với các ngân hàng Việt Nam, việc tham gia xu hướng này là tất yếu để nâng cao năng lực cạnh tranh và cải thiện lợi nhuận. Tuy nhiên, để tận dụng tối đa lợi thế của Big Data, cần chuẩn bị đồng bộ các yếu tố then chốt: chính sách, vốn, con người và công nghệ

1.6 Cơ hội và thách thức của các doanh nghiệp khi sử dụng Big Data

1.6.1 Cơ hội

HADOOP

Tiếp cận và nghiên cứu về dữ liệu lớn giúp chúng ta có thêm phương án giải quyết, xử lý, và đối phó với những thách thức sản xuất số liệu thống kê chính thức trong hiện tại và tương lai. Cần tiến hành những nghiên cứu thực nghiệm để khám phá những ứng dụng tiềm năng của dữ liệu lớn.

Tiếp cận và nghiên cứu về dữ liệu lớn giúp chúng ta có những văn bản pháp lý bổ sung có thể giúp cho các cơ quan thống kê chính thức có điều kiện để thực hiện được khai thác dữ liệu thông qua hồ sơ hành chính, ngoài ra dữ liệu cũng được đảm bảo và giữ bí mật những văn bản pháp lý bổ sung này

Sử dụng dữ liệu lớn đem lại niềm tin của cộng đồng với thống kê chính thức do quá trình sản xuất số liệu thống kê chính thức với dữ liệu lớn hoàn toàn không có sự tác động chú ý của con người

1.6.2 Thách thức

Về Tài chính: Khó ước lượng chi phí triển khai, yêu cầu đầu tư lớn vào hạ tầng và hệ thống (như Hadoop, hệ thống phân tán), nên nhiều tổ chức khó triển khai thành công.

Về Chính sách và pháp lý: Việc truy cập và khai thác dữ liệu phụ thuộc vào luật từng quốc gia dẫn đến hạn chế chia sẻ và sử dụng dữ liệu (ví dụ khác biệt giữa các nước).

Về Nhân lực và quản lý dữ liệu: Thiếu nhân lực chất lượng cao; sự phối hợp giữa tổ chức tư nhân và nhà nước còn hạn chế dẫn đến khó khai thác hiệu quả.

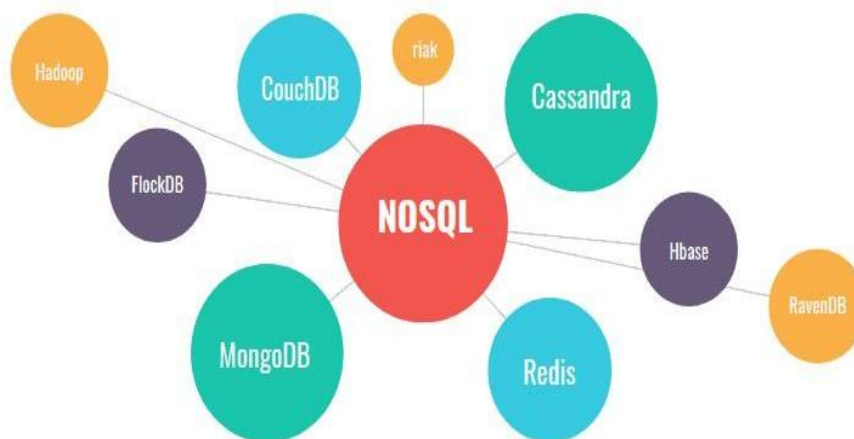
Về Hạ tầng công nghệ thông tin: thách thức lớn về kỹ thuật. Cần cải thiện tốc độ truy cập, chuẩn hóa API, tích hợp với hệ thống truyền thống.

Phần 2 . NoSQL

2.1 Định nghĩa

Cơ sở dữ liệu NoSQL là một Hệ thống quản lý dữ liệu không quan hệ (non-relational Data Management System) có lược đồ (schema) linh hoạt. Nó dễ mở rộng. Mục đích chính của việc sử dụng cơ sở dữ liệu NoSQL là dành cho các kho dữ liệu phân tán với nhu cầu lưu trữ dữ liệu lớn. NoSQL được sử dụng cho Dữ liệu lớn và ứng dụng web thời gian thực. Chẳng hạn các công ty như Twitter, Facebook và Google thu thập hàng terabyte dữ liệu người dùng mỗi ngày.

Cơ sở dữ liệu NoSQL là viết tắt của “Not Only SQL” hoặc “Not SQL”. Với cơ sở dữ liệu quan hệ RDBMS ta sử dụng cú pháp SQL để lưu trữ và truy xuất dữ liệu. Đối với hệ thống cơ sở dữ liệu NoSQL, nó bao gồm một loạt các công nghệ cơ sở dữ liệu có thể lưu trữ dữ liệu có cấu trúc, bán cấu trúc, phi cấu trúc và đa hình.



Hình 5. Các cơ sở dữ liệu NoSQL

2.2 Ưu - nhược điểm của cơ sở dữ liệu NoSQL

2.2.1 Ưu điểm

Các ứng dụng hiện đại phải đối mặt với một số thách thức có thể được giải quyết bằng cơ sở dữ liệu NoSQL. Ví dụ, các ứng dụng xử lý một khối lượng dữ liệu lớn từ các nguồn khác nhau như phương tiện truyền thông xã hội, cảm biến thông minh và cơ sở dữ liệu của bên thứ ba. Tất cả dữ liệu khác nhau này không phù hợp với mô hình quan hệ. Việc thực thi các cấu trúc dạng bảng có thể dẫn đến dư thừa, sao chép dữ liệu và các vấn đề về hiệu suất trên quy mô lớn.

Cơ sở dữ liệu NoSQL chuyên dành cho các mô hình dữ liệu phi quan hệ và có sơ đồ linh hoạt để xây dựng các ứng dụng hiện đại. Chúng được công nhận rộng rãi vì khả năng dễ phát triển, chức năng cũng như hiệu năng ở quy mô lớn. Lợi ích của cơ sở dữ liệu NoSQL được liệt kê dưới đây.

- **Tính linh hoạt** : Cơ sở dữ liệu NoSQL thường cung cấp các sơ đồ linh hoạt giúp công đoạn phát triển nhanh hơn và có khả năng lặp lại cao hơn. Mô hình dữ liệu linh hoạt biến cơ sở dữ liệu NoSQL thành lựa chọn lý tưởng cho dữ liệu không được tổ chức thành cấu trúc hoặc có cấu trúc chưa hoàn chỉnh

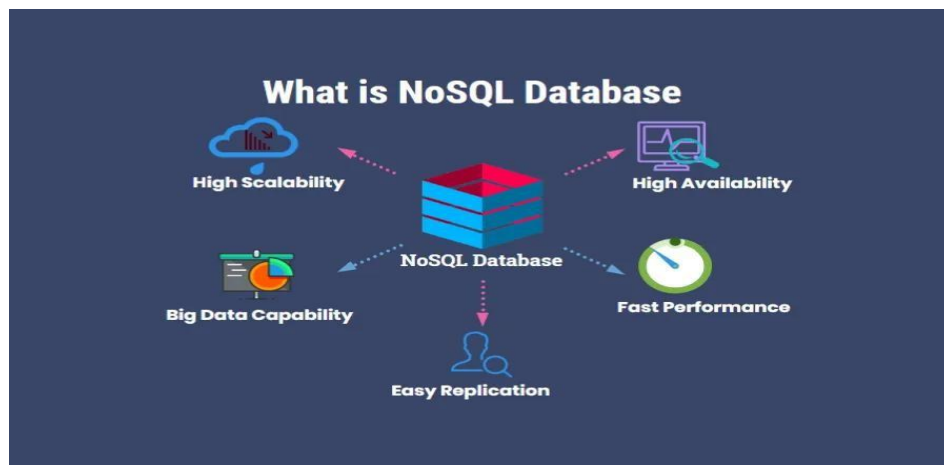
HADOOP

-Khả năng mở rộng : Cơ sở dữ liệu NoSQL thường được thiết kế để tăng quy mô bằng cách sử dụng các cụm phần cứng được phân phối thay vì tăng quy mô bằng cách bổ sung máy chủ mạnh và tốn kém. Một số nhà cung cấp dịch vụ đám mây xử lý các hoạt động này một cách không công khai dưới dạng dịch vụ được quản lý đầy đủ.

-Hiệu năng cao: Cơ sở dữ liệu NoSQL được tối ưu hóa cho các mô hình dữ liệu cụ thể và các mẫu truy cập. Chúng cho phép hiệu suất cao hơn so với khi bạn đang cố gắng thực hiện chức năng tương tự với cơ sở dữ liệu quan hệ

-Chức năng cao: Cơ sở dữ liệu NoSQL cung cấp các API và kiểu dữ liệu cực kỳ thiết thực được xây dựng riêng cho từng mô hình dữ liệu tương ứng

-Khả năng chịu lỗi cao : NoSQL thường được thiết kế để hoạt động tốt trên các hệ thống phân tán, giúp dữ liệu vẫn được truy cập bình thường ngay cả khi một số nút trong hệ thống bị lỗi. Cơ chế sao lưu và nhân bản dữ liệu (replication) giúp đảm bảo tính sẵn sàng và độ tin cậy.



Hình 6. Ưu điểm của cơ sở dữ liệu NoSQL

2.2.2 Nhược điểm

Bên cạnh những ưu điểm của nó thì NoSQL Database cũng có những nhược điểm sau:

-Quản lý dữ liệu: Mục đích của các công cụ dữ liệu lớn là làm cho việc quản lý một lượng

lớn dữ liệu trở nên đơn giản nhất. Nhưng quản lý dữ liệu trong NoSQL phức tạp hơn nhiều so với cơ sở dữ liệu quan hệ. Đặc biệt, NoSQL nổi tiếng là khó cài đặt và thậm chí là để quản lý nó hàng ngày cũng tốn khá nhiều thời gian.

- Sao lưu dữ liệu: Sao lưu là một điểm yếu lớn đối với một số cơ sở dữ liệu NoSQL như MongoDB. Nó không có cách tiếp cận để làm sao lưu dữ liệu một cách nhất quán.

Thiếu tính nhất quán: NoSQL đánh đổi sự nhất quán để ưu tiên tốc độ, hiệu suất hiệu quả hơn.

Trọng tâm hẹp: Cơ sở dữ liệu NoSQL có trọng tâm rất hẹp vì nó chủ yếu được thiết kế để

lưu trữ nhưng nó cung cấp rất ít chức năng.

-Mã nguồn mở: NoSQL là cơ sở dữ liệu mã nguồn mở và không có tiêu chuẩn đáng tin cậy cho NoSQL được nêu ra.

-Không có lược đồ: Ngay cả khi bạn lấy dữ liệu ở dạng tự do, bạn hầu như luôn cần áp đặt các ràng buộc để làm cho nó hữu ích. Với NoSQL, trách nhiệm sẽ được chuyển từ cơ sở dữ liệu sang nhà phát triển, lập trình ứng dụng.

-Kỹ năng NoSQL: Một hạn chế khác đối với NoSQL là người sử dụng có thể sẽ thiếu các kỹ năng chuyên môn ở mức tương đối vì hệ thống này còn khá mới và không phải ai cũng biết sử dụng nó một cách thành thạo.

2.3 Phân loại cơ sở dữ liệu NoSQL

NoSQL cung cấp nhiều cách khác nhau trong tổ chức dữ liệu. Với việc hỗ trợ nhiều cấu trúc dữ liệu đa dạng, NoSQL có thể được áp dụng trong việc phân tích dữ liệu, quản lý dữ liệu lớn, mạng xã hội và phát triển ứng dụng di động. Cơ sở dữ liệu NoSQL có khả năng quản lý dữ liệu thông qua các mô hình quản lý dữ liệu sau đây.

2.3.1 Cơ sở dữ liệu Key-Value

Cơ sở dữ liệu Key-value thuộc loại cơ sở dữ liệu NoSQL, mà ở đó mỗi mục (item) được tổ chức và lưu trữ dưới dạng cặp key-value. Mỗi đối tượng dữ liệu được xác định bằng một khóa (key) duy nhất và giá trị (value) tương ứng chứa thông tin liên quan đến khóa đó. Trong mô hình này, key thường được hạn chế ở dạng chuỗi (string), trong khi value có thể mang nhiều định dạng khác nhau như chuỗi, JSON, XML, hoặc các kiểu dữ liệu phức tạp tùy thuộc vào khả năng hỗ trợ của hệ thống cụ thể. Sự linh hoạt này cho phép xử lý lượng dữ liệu khổng lồ.

Cơ sở dữ liệu Key-Value là hệ thống lưu trữ dữ liệu theo dạng cặp index key-value và truy vấn value bằng cách sử dụng index key. Cơ sở dữ liệu Key-value lưu trữ dữ liệu giống với một cơ sở dữ liệu quan hệ (RDMS) nhưng các bảng của cơ sở dữ liệu key-value chỉ có 2 cột.

Một số cơ sở dữ liệu Key-Value cho phép thực hiện các phép nối (join) cơ bản, nhưng có thể không phù hợp với các phép nối phức tạp.

Có nhiều loại cơ sở dữ liệu Key-Value khác nhau, với sự khác biệt chủ yếu về cách xử lý theo định lý CAP (Consistency, Availability, Partition Tolerance) và cấu hình sử dụng bộ nhớ so với lưu trữ. Cơ sở dữ liệu này có hiệu suất truy vấn nhanh, thích hợp cho các ứng dụng yêu cầu content caching (lưu trữ dữ liệu), chẳng hạn như trang web thương mại điện tử thường xuyên cập nhật danh sách top 10 sản phẩm phổ biến.

Các đặc điểm của cơ sở dữ liệu key-value:

- Nhất quán (Consistency)
- Hỗ trợ giao dịch (Transaction)
- Tính năng truy vấn (Query Feature)
- Cấu trúc dữ liệu (Data Structure)

HADOOP

-Mở rộng (Scaling)

Một số cơ sở dữ liệu Key-Value phổ biến bao gồm Dynamo DB, Redis và BerkleyDB.

```
1 Key: user:12345
2 Value: {"name": "foo bar", "email": "foo@bar.com", "designation": "software
  developer"}
```

Hình 7. Cơ sở dữ liệu Key-Value

Ưu và nhược điểm của cơ sở dữ liệu Key-Value

Ưu điểm	Khuyết điểm
• Mô hình dữ liệu đơn giản • Dễ dàng mở rộng (scalability) • Giá trị có thể là XML, JSON hoặc các schema linh hoạt • Tốc độ nhanh • Phù hợp cho các trường hợp dữ liệu không có quan hệ (phi quan hệ)	• Không tự hỗ trợ liên kết các dữ liệu giữa các cặp key-value, phải tự tạo khóa ngoại (foreign key) riêng • Không phù hợp với dữ liệu phức tạp • Thiếu khả năng quét (Scanning) • Chỉ phù hợp với các lệnh CRUD (create, read, update, delete)

2.3.2 Cơ sở dữ liệu Document

Cơ sở dữ liệu Document là một sự mở rộng của cơ sở dữ liệu Key-value, nơi dữ liệu được tổ chức trong các tài liệu có cấu trúc như JSON hoặc XML. Các tài liệu này giúp dễ dàng ánh xạ các đối tượng (Object) trong phần mềm hướng đối tượng (Oriented Object software).

Cơ sở dữ liệu Document không yêu cầu phải xác định trước schema và cho phép lưu trữ dữ liệu phức tạp ở định dạng tài liệu như JSON hoặc XML. Mỗi tài liệu trong cơ sở dữ liệu Document là độc lập và không thể thực hiện các phép nối (join) mà trong ngữ cảnh này có nghĩa là không có cơ chế tự động để duy trì tính toàn vẹn quan hệ giữa các phần dữ liệu khi thực hiện các phép nối.

Mặc dù không hỗ trợ dữ liệu quan hệ và bên cạnh các ưu điểm tương tự như cơ sở dữ liệu key-value, cơ sở dữ liệu Document có những ưu điểm bổ sung như khả năng truy vấn không chỉ bằng key mà còn truy vấn thuộc tính (attribute) trong tài liệu. Dữ liệu trong mỗi tài liệu có thể ở định dạng khác nhau và có thể được truy vấn một cách linh hoạt.

Ví dụ, một ứng dụng quản lý thông tin cá nhân, trong đó mỗi người dùng được biểu diễn bằng một tài liệu. Mỗi tài liệu có thể chứa thông tin như tên, địa chỉ, số điện thoại, và sở thích. Mỗi người dùng có thể có một số lượng tài liệu không giới hạn, và mỗi tài liệu có thể được cập nhật độc lập mà không làm ảnh hưởng đến các tài liệu khác. Điều này giúp quản lý thông tin cá nhân một cách linh hoạt và thuận tiện.

HADOOP

Một số cơ sở dữ liệu Document phổ biến là MongoDB, CouchDB và Lotus Notes.

```
1  {
2    "_id": "12345",
3    "name": "foo bar",
4    "email": "foo@bar.com",
5    "address": {
6      "street": "123 foo street",
7      "city": "some city",
8      "state": "some state",
9      "zip": "123456"
10   },
11   "hobbies": ["music", "guitar", "reading"]
12 }
```

Hình 8. Cơ sở dữ liệu Document

Ưu và nhược điểm cơ sở dữ liệu Document

Ưu điểm	Khuyết điểm
• Mô hình dữ liệu đơn giản và mạnh mẽ • Định dạng mở • Có thể mở rộng (scalability) • Không có khóa ngoại (foreign keys)	• Chỉ là giải pháp tạm thời để lưu trữ dữ liệu dạng bảng và không hiệu quả khi xử lý các bảng lớn • Truy vấn giới hạn ở các key và index • Cần sử dụng phương pháp MapReduce khi thực hiện các truy vấn phức tạp

2.3.3 Cơ sở dữ liệu Column-Family

Cơ sở dữ liệu Column Family kết hợp đặc điểm khai báo của cơ sở dữ liệu quan hệ với đặc điểm key-value pair và free schema của cơ sở dữ liệu Key-value. Thay vì lưu trữ dữ liệu dưới dạng hàng như cơ sở dữ liệu quan hệ, Column Family lưu trữ chúng dưới dạng các cột.

Cơ sở dữ liệu Column Family được biểu diễn như một hệ thống lưu trữ Key-value đa chiều, tương tự như một bản đồ hoặc một mảng kết hợp (associative array). Dữ liệu trong cơ sở dữ liệu này không chỉ bền vững, tức là giá trị vẫn tồn tại sau khi được tạo ra hoặc truy cập (access), mà còn được phân tán trên nhiều nút (node) tính toán và lưu trữ, giúp tối ưu hóa hiệu suất và khả năng mở rộng. Các key được sắp xếp để hỗ trợ quá trình truy vấn, và dữ liệu có thể bị khuyết như trong cơ sở dữ liệu quan hệ.

Có hai loại Column Family:

-Standard Column Family: gồm một cặp key-value, trong đó key được ánh xạ tới một giá trị (value) là một tập hợp các cột.

-Super Column Family: bao gồm một cặp key-value, trong đó key được ánh xạ tới một giá trị (value) là các column family. Tương tự như trong cơ sở dữ liệu quan hệ, super column family có thể được coi là một “view” trên một loạt các bảng. Nó cũng có thể được hiểu như một bản đồ của các bảng.

Các đặc điểm của cơ sở dữ liệu Column Family:

- Lưu trữ key đa chiều (Multidimensional key store)
- Bền vững
- Phân tán
- Khả năng linh hoạt cao

Một số cơ sở dữ liệu Column Family phổ biến bao gồm Google’s Bigtable, Cassandra, và HBase.

name	id	email	dob	city
Foo bar	12345	foo@bar.com		Some city
Carn Yale	34521	bar@foo.com	12-05-1972	

Hình 9. Cơ sở dữ liệu Column-Family

Ưu và nhược điểm cơ sở dữ liệu Column Family

Ưu điểm	Khuyết điểm
- Hỗ trợ dữ liệu bán cấu trúc - Tự động index - Có thể mở rộng (scalability)	Chỉ là giải pháp tạm thời để lưu trữ dữ liệu dạng bảng và không hiệu quả khi xử lý các bảng lớn.

2.3.4 Cơ sở dữ liệu Graph

Cơ sở dữ liệu Graph là hệ thống lưu trữ dữ liệu theo hướng đồ thị. Một cơ sở dữ liệu

HADOOP

Graph phải có tính năng truy xuất dữ liệu không cần index. Thay vào đó mỗi nút chứa một con trỏ (pointer) trực tiếp đến các nút liền kề của nó. Khi số lượng node tăng, chi phí để di chuyển từ nút này sang nút khác (hay còn gọi là một “bước nhảy”) vẫn được duy trì ổn định.

Các cơ sở dữ liệu Graph được tối ưu hóa để duyệt qua dữ liệu kết nối, ví dụ như việc tìm kiếm “Bạn bè chung” trên mạng xã hội có thể được thực hiện một cách nhanh chóng và hiệu quả bằng cách duyệt qua các mối quan hệ kết nối giữa các người dùng và sở thích chung mà không cần sử dụng index.

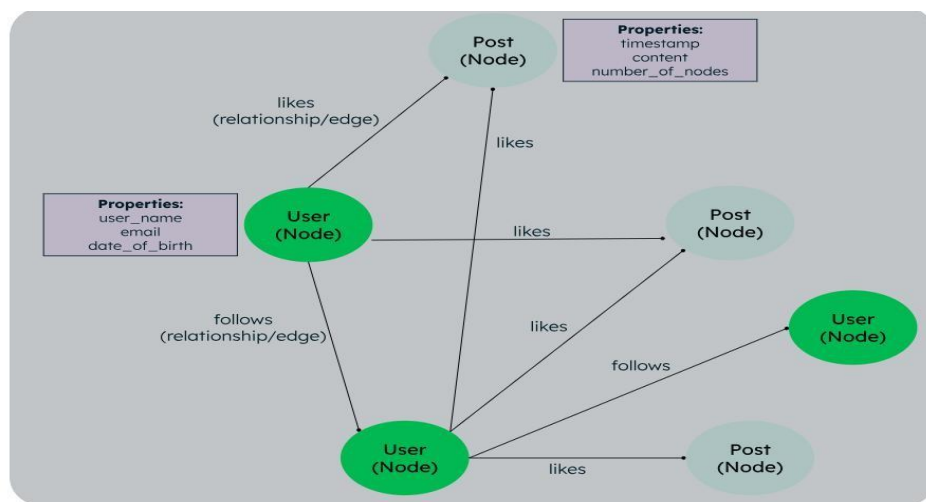
Các đặc điểm của cơ sở dữ liệu Graph:

-Linh hoạt

-Tốc độ nhanh

-Cải thiện hiệu suất ngay cả khi phải xử lý lượng dữ liệu khổng lồ

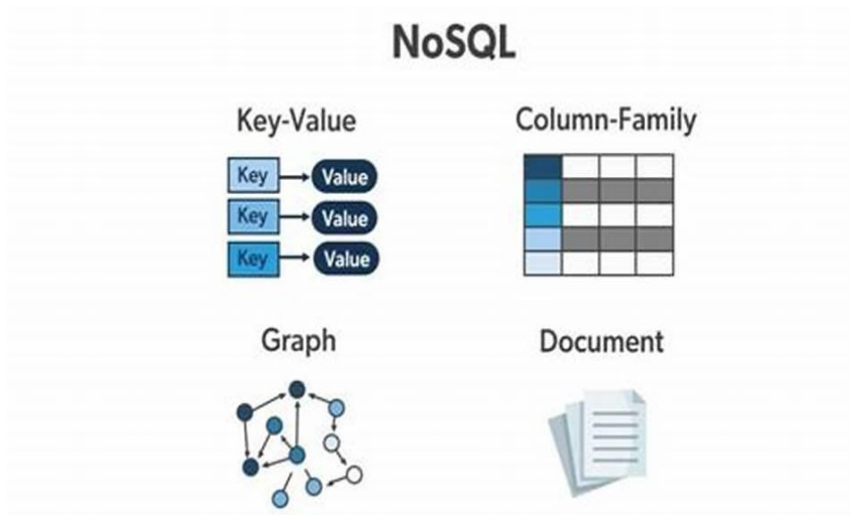
Một số cơ sở dữ liệu Graph phổ biến gồm Neo4j, OrientDB và Allegrograph.



Hình 10. Cơ sở dữ liệu Graph

Ưu và nhược điểm cơ sở dữ liệu Graph

Ưu điểm	Khuyết điểm
• Là một công cụ mạnh mẽ • Index được lưu local gần data, dễ dàng truy xuất • Có thể đáp ứng ACID (Atomicity, Consistency, Isolation, Durability) • Cấu trúc linh hoạt • Khả năng cung cấp kết quả ngay lập tức (in real-time)	• Khó mở rộng chiều ngang (thêm máy chủ), mặc dù có thể mở rộng chiều dọc (nâng cấp máy chủ).



Hình 11. Hình ảnh về phân loại các CSDL NoSQL

2.4 So sánh NoSQL với SQL

Tiêu chí	SQL	NoSQL
Loại hình	SQL databases là cơ sở dữ liệu dựa trên bảng và các bảng này có quan hệ với nhau.	NoSQL databases có thể dựa trên nhiều hướng lưu trữ khác nhau: document-oriented, column-oriented, graph-based,...
Ngôn ngữ truy vấn	Sử dụng SQL để truy vấn và quản lý dữ liệu. Các câu lệnh như SELECT, INSERT, UPDATE, DELETE được sử dụng để tương tác với dữ liệu.	Ngôn ngữ truy vấn đa dạng. Các cơ sở dữ liệu NoSQL sử dụng các ngôn ngữ truy vấn không phải là SQL.
Khả năng mở rộng	Cơ sở dữ liệu SQL có thể được thu nhỏ theo chiều dọc, được mở rộng bằng cách tăng lưu lượng phần cứng.	Được tùy biến theo chiều ngang, mở rộng bằng cách tăng số lượng máy chủ cơ sở dữ liệu.
Mục đích sử dụng	Được thiết kế dành cho các ứng dụng xử lý giao dịch trực tuyến, trong giao dịch có độ ổn định cao và thích hợp để xử lý phân tích trực tuyến.	Được thiết kế phục vụ cho việc phân tích dữ liệu có cấu trúc chưa hoàn chỉnh.

HADOOP

Lựa chọn sử dụng	- Dữ liệu có mối quan hệ phức tạp. - Yêu cầu về tính ACID và tính toàn vẹn dữ liệu cao. - Lược đồ cố định, được định rõ trước.	- Dữ liệu bán cấu trúc và phi cấu trúc. - Khả năng mở rộng nhanh chóng. - Hiệu suất cao cho đọc và ghi. - Lược đồ động, linh hoạt và có thể thay đổi.
Mã nguồn	Kết hợp của các mã nguồn mở như Postgres & MySQL và thương mại như Oracle Database.	Open-source.
Hiệu suất	Hiệu suất thường tốt cho các truy vấn liên quan đến mối quan hệ giữa các bảng.	Hiệu suất cao trong việc truy cập và cập nhật toàn bộ document hoặc các trường cụ thể trong document.
ACID- BASE	ACID(Atomicity, Consistency, Isolation, Durability) là một chuẩn cho RDBMS	BASE (Basically Available, Soft State, Eventually Consistent) là một mô hình của nhiều hệ thống NoSQL
Ưu điểm	- Hỗ trợ ACID (Atomicity, Consistency, Isolation, Durability), đảm bảo tính toàn vẹn và nhất quán của dữ liệu. - Truy vấn dữ liệu dễ dàng và linh hoạt với ngôn ngữ truy vấn SQL chuẩn. - Dữ liệu có cấu trúc, giúp giảm thiểu việc lặp lại thông tin và tối ưu hóa hiệu suất truy vấn.	- Hỗ trợ quy mô lớn và dễ dàng mở rộng hệ thống. - Được thiết kế để xử lý dữ liệu bán và phi cấu trúc và linh hoạt trong việc thay đổi cấu trúc dữ liệu. - Hiệu suất cao khi truy vấn dữ liệu lớn và phức tạp.
Nhược điểm	- Hạn chế trong việc mở rộng quy mô lớn và xử lý dữ liệu phi cấu trúc. - Khó khăn trong việc thay đổi cấu trúc dữ liệu khi ứng dụng phát	- Không hỗ trợ đầy đủ các tính năng ACID, có thể dẫn đến sự không nhất quán và toàn vẹn trong một số tình huống. - Cú pháp truy vấn và tương tác

	triển.	với dữ liệu phức tạp hơn so với SQL.
--	--------	--------------------------------------

2.5 Kết luận

NoSQL database chỉ là một loại database có cách truy vấn, lưu trữ dữ liệu hoàn toàn khác so với RDBMS và SQL. Các ứng dụng hiện nay vẫn cần cả hai loại cơ sở dữ liệu trên để đáp ứng như cầu tùy theo từng lĩnh vực khác nhau.

Phần 3 : HADOOP

3.1 Hadoop là gì?

Hadoop là một Apache framework mã nguồn mở cho phép phát triển các ứng dụng phân tán (distributed processing) để lưu trữ và quản lý các tập dữ liệu lớn. Hadoop hiện thực mô hình MapReduce, mô hình mà ứng dụng sẽ được chia nhỏ ra thành nhiều phân đoạn khác nhau được chạy song song trên nhiều node khác nhau. Hadoop được viết bằng Java tuy nhiên vẫn hỗ trợ C++, Python, Perl bằng cơ chế streaming.

3.2 Lịch sử phát triển và các phiên bản của Hadoop

Hadoop bắt đầu từ một dự án mang tên Nutch, được phát triển bởi Doug Cutting và Mike Cafarella vào năm 2002 tại Yahoo!. Nutch là một công cụ tìm kiếm, nhưng vào năm 2004, nhóm phát triển nhận thấy cần phải xây dựng một hệ thống lưu trữ và xử lý dữ liệu phân tán hiệu quả hơn. Điều này dẫn đến sự phát triển của MapReduce và HDFS (Hadoop Distributed File System), là hai thành phần cốt lõi của Hadoop.

Ra mắt Hadoop (2006)

- 2006: Doug Cutting và Mike Cafarella quyết định tách ra từ dự án Nutch để phát triển một dự án độc lập và mã nguồn mở có tên gọi Hadoop. Hadoop được lấy tên từ một chú voi đồ chơi của con trai Doug Cutting.
- 2006-2008: Hadoop bắt đầu được phát triển và triển khai tại Yahoo! để xử lý và lưu trữ lượng dữ liệu lớn cho công ty. Hadoop dựa trên hai công nghệ cơ bản: MapReduce (cho việc xử lý dữ liệu) và HDFS (cho việc lưu trữ dữ liệu phân tán).

Tăng trưởng và phát triển (2008-2012)

- 2008: Apache Hadoop trở thành một dự án của Apache Software Foundation, giúp Hadoop phát triển nhanh chóng nhờ sự đóng góp của cộng đồng mã nguồn mở.
- 2009: Yahoo! sử dụng Hadoop để xử lý hàng triệu dữ liệu và báo cáo thành công. Hadoop trở thành lựa chọn phổ biến trong cộng đồng doanh nghiệp và các công ty công nghệ lớn.
- 2011: Hadoop v1 được phát hành với nhiều cải tiến về khả năng mở rộng và quản lý dữ liệu phân tán.

Hadoop v2 và YARN (2012-2014)

- 2012: Hadoop v2 ra mắt, đánh dấu một bước ngoặt quan trọng với sự ra đời của YARN (Yet Another Resource Negotiator), một hệ thống quản lý tài nguyên giúp Hadoop trở nên linh hoạt hơn và có thể hỗ trợ nhiều mô hình tính toán khác nhau ngoài MapReduce. 2013: Hadoop tiếp tục phát triển mạnh mẽ và bắt đầu hỗ trợ các công cụ khác như Hive, Pig, HBase, và Sqoop, tạo nên một hệ sinh thái dữ liệu mạnh mẽ và toàn diện.

Hadoop và việc áp dụng rộng rãi trong doanh nghiệp (2014-2018)

- 2014-2016: Hadoop trở thành công cụ phổ biến trong các doanh nghiệp lớn như

HADOOP

Facebook, LinkedIn, và Microsoft để xử lý dữ liệu lớn (Big Data). Các công ty này sử dụng Hadoop để phân tích hành vi người dùng, tối ưu hóa quy trình và cải thiện dịch vụ khách hàng.

- 2015: Hadoop tiếp tục phát triển mạnh mẽ với các phiên bản mới, và hệ sinh thái Hadoop mở rộng ra với các công cụ như Apache Spark, Apache Flink, Apache Kafka, và Apache Storm, cung cấp khả năng xử lý dữ liệu theo thời gian thực và phân tích dữ liệu nhanh chóng.

Hadoop 3.x và các cải tiến hiện đại (2018 - nay)

-2017: Hadoop 3.0 được phát hành với nhiều cải tiến quan trọng, bao gồm:

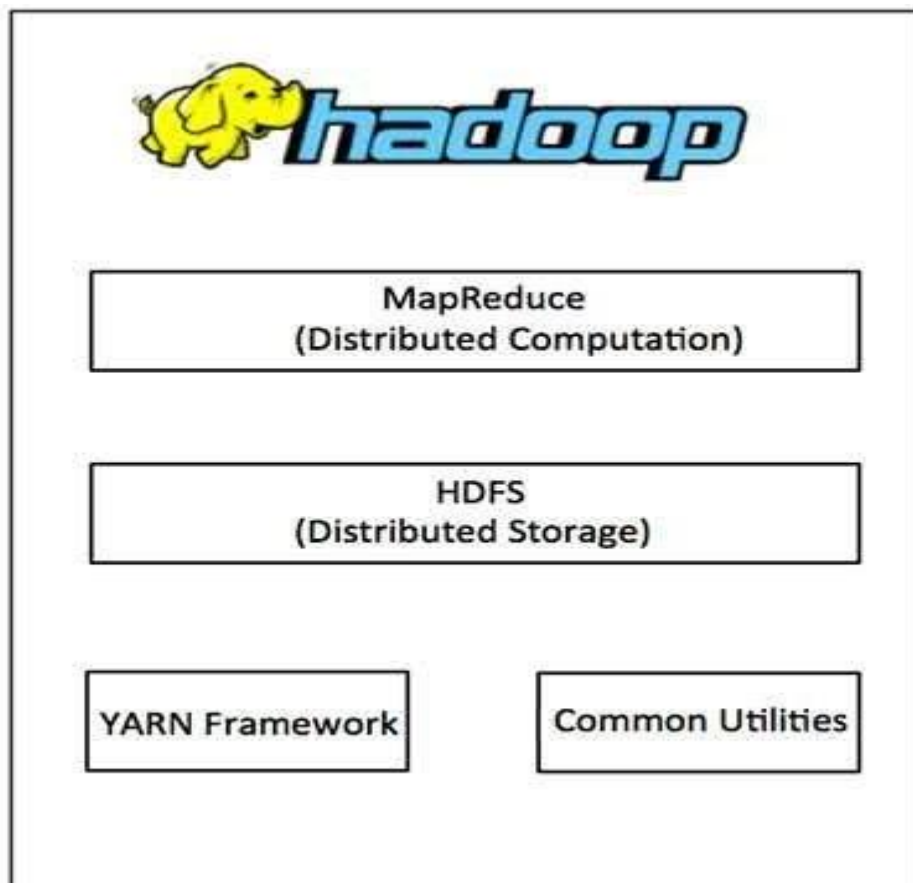
-HDFS Erasure Coding: Giúp giảm chi phí lưu trữ và tăng khả năng phục hồi dữ liệu.

-YARN Federation: Cho phép mở rộng Hadoop bằng cách hỗ trợ nhiều cụm (clusters) và tăng cường tính sẵn sàng.

-Hadoop 3.x cũng hỗ trợ các tính năng như Kubernetes và các công nghệ container hóa giúp dễ dàng triển khai Hadoop trên đám mây.

3.3 Kiến trúc tổng quan của Hadoop

Một cụm Hadoop nhỏ gồm 1 master node và nhiều worker/slave node. Toàn bộ cụm chứa 2 lớp, một lớp MapReduce Layer và lớp kia là HDFS Layer. Mỗi lớp có các thành phần liên quan riêng. Master node gồm JobTracker, TaskTracker, NameNode, và DataNode. Slave/worker node gồm DataNode, và TaskTracker. Cũng có thể slave/worker node chỉ là dữ liệu hoặc node để tính toán.



HADOOP

Hình 12. Tổng quan về HADOOP

Hadoop framework gồm 4 module:

Hadoop Distributed File System (HDFS)

Đây là hệ thống file phân tán cung cấp truy cập thông lượng cao cho ứng dụng khai thác dữ liệu. Hadoop Distributed File System (HDFS) là hệ thống tập tin ảo. Khi chúng ta di chuyển 1 tập tin trên HDFS, nó tự động chia thành nhiều mảnh nhỏ. Các đoạn nhỏ của tập tin sẽ được nhân rộng và lưu trữ trên nhiều máy chủ khác để tăng sức chịu lỗi và tính sẵn sàng cao.

HDFS sử dụng kiến trúc master/slave, trong đó master gồm một NameNode để quản lý hệ thống file metadata và một hay nhiều slave DataNodes để lưu trữ dữ liệu thực tại.

Một tập tin với định dạng HDFS được chia thành nhiều khối và những khối này được lưu trữ trong một tập các DataNodes. NameNode định nghĩa ánh xạ từ các khối đến các DataNode. Các DataNode điều hành các tác vụ đọc và ghi dữ liệu lên hệ thống file. Chúng cũng quản lý việc tạo, huỷ, và nhân rộng các khối thông qua các chỉ thị từ NameNode.

Hadoop MapReduce

Đây là hệ thống dựa trên YARN dùng để xử lý song song các tập dữ liệu lớn. Là cách chia

một vấn đề dữ liệu lớn hơn thành các đoạn nhỏ hơn và phân tán nó trên nhiều máy chủ. Mỗi máy chủ có 1 tập tài nguyên riêng và máy chủ xử lý dữ liệu trên cục bộ. Khi máy chủ xử lý xong dữ liệu, chúng sẽ gọi trở về máy chủ chính.

MapReduce gồm một single master (máy chủ) JobTracker và các slave (máy trạm) TaskTracker trên mỗi cluster-node. Master có nhiệm vụ quản lý tài nguyên, theo dõi quá trình tiêu thụ tài nguyên và lập lịch quản lý các tác vụ trên các máy trạm, theo dõi chúng và thực thi lại các tác vụ bị lỗi. Những máy slave TaskTracker thực thi các tác vụ được master chỉ định và cung cấp thông tin trạng thái tác vụ (task-status) để master theo dõi.

JobTracker là một điểm yếu của Hadoop Mapreduce. Nếu JobTracker bị lỗi thì mọi công việc liên quan sẽ bị ngắt quãng.

Hadoop Common

Đây là các thư viện và tiện ích cần thiết của Java để các module khác sử dụng. Những thư viện này cung cấp hệ thống file và lớp OS trừu tượng, đồng thời chứa các mã lệnh Java để khởi động Hadoop.

Hadoop YARN

Quản lý tài nguyên của các hệ thống lưu trữ dữ liệu và chạy phân tích

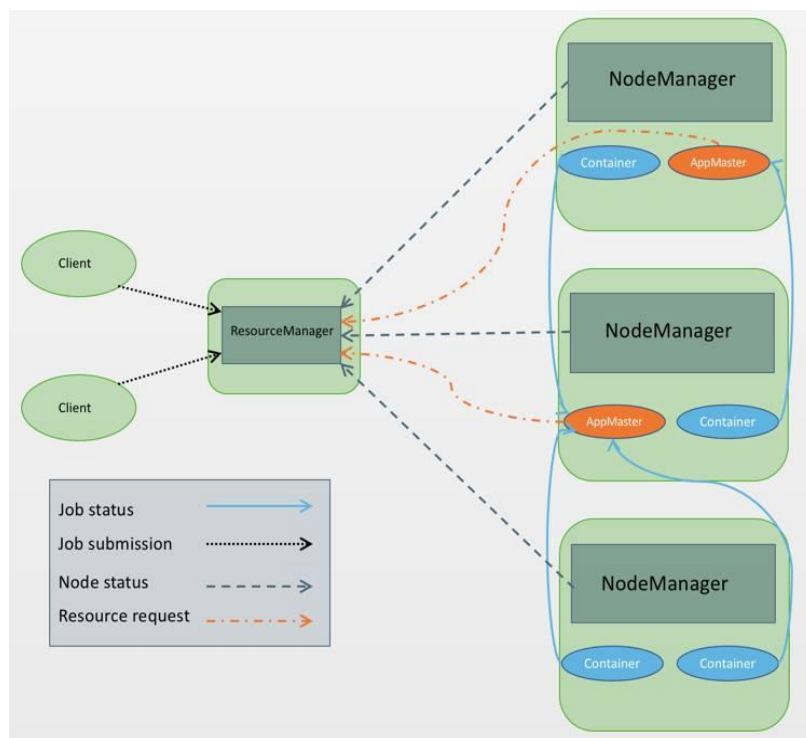
Phần 4. Yarn (Yet Another Resource Negotiator)

Yarn là công cụ quản lý gói phần mềm (package) JavaScript mã nguồn mở cho phép người dùng sử dụng và chia sẻ các gói phần mềm do bạn viết với các lập trình viên trên toàn thế giới.

Đây là dự án bắt đầu được viết từ tháng 1/2016, sau quãng thời gian thử nghiệm và hoàn thiện đã được mở ra thành mã nguồn mở, và thật không có gì ngạc nhiên khi với những tính năng nổi trội vượt bậc đã có 10.000 stars chỉ trong 1 ngày tại Github. Chứng tỏ sự thành công và là tín hiệu thay thế rõ ràng NPM

4.1 Kiến trúc và cơ chế hoạt động của Yarn

YARN có 2 chức năng cơ bản là quản lý tài nguyên và theo dõi, lập lịch công việc. Thực hiện hai nhiệm vụ này tương ứng là ResourceManager và ApplicationMaster. Các thành phần chính của YARN bao gồm ResourceManager, NodeManager và ApplicationMaster.



Hình 13. Kiến trúc của YARN

ResourceManager có nhiệm vụ quản lý toàn bộ tài nguyên trong cụm, nó cung cấp tài nguyên là core, RAM cho ứng dụng cần dùng, đồng thời cũng là đầu não quản lý các NodeManager. Trong một namespace chỉ có một active ResourceManager tại một thời điểm.

ResourceManager có hai thành phần chính là Scheduler và ApplicationManager.

- **Scheduler:** có nhiệm vụ cung cấp tài nguyên cho các ứng dụng sử dụng, nó là thuần scheduler và không có nhiệm vụ quản lý hay theo dõi tình trạng của ứng dụng, hoặc như việc restart các task bị fail. Việc cung cấp tài nguyên là dựa trên yêu cầu của ứng dụng, các tài nguyên như RAM, core,.. Việc này cũng dựa trên các chính sách chia nhỏ tài nguyên thành các queue, các chính sách như CapacityScheduler và FairScheduler.

- **ApplicationManager:** Có nhiệm vụ tiếp nhận các job từ client, cấp phát tài nguyên để start ApplicationMaster cho từng ứng dụng, đồng thời cung cấp dịch vụ restart ApplicationMaster trong trường hợp lỗi.

NodeManager được cài trên 1 node có nhiệm vụ quản lý tài nguyên (RAM, core...) và job trên node đó. NodeManager thường xuyên gửi heartbeat đến RM thông báo tình trạng, mục đích chính của NM là quản lý các container mà RM giao cho nó, kill các container này nếu RM yêu cầu.

ApplicationMaster là một container được khởi tạo bởi RM, tương ứng với mỗi job submit lên RM là một ApplicationMaster. AppMaster có nhiệm vụ điều phối các hoạt động của một job trong cụm, xin cấp phát tài nguyên từ ResourceManager và kết nối với NodeManger để quản lý các container.

Container là tiến trình của YARN, tập hợp các tài nguyên vật lý như core, RAM. YARN container được quản lý bởi đối tượng Container Launch Context, đối tượng này lưu trữ các thông tin như biến môi trường, dependencies, security tokens và các chỉ thị để tạo tiến trình. Container cấp quyền cho các ứng dụng chạy trên nó sử dụng một lượng tài nguyên nhất định.

Cơ chế hoạt động

Cách hoạt động của YARN diễn ra theo các bước sau:

1. Người dùng gửi ứng dụng đến ResourceManager.
2. ResourceManager cấp một container để khởi chạy ApplicationMaster.
3. ApplicationMaster phân tích ứng dụng và yêu cầu thêm tài nguyên.
4. ResourceManager phân bổ container trên các NodeManager phù hợp.
5. NodeManager thực thi các task trong container.
6. ApplicationMaster theo dõi tiến trình và xử lý lỗi.
7. Khi hoàn thành, tài nguyên được giải phóng và kết quả trả về.

4.2 Cơ chế lập lịch tài nguyên trong Yarn (FIFO, Capacity Scheduler, Fair Scheduler)

Trong Apache Hadoop YARN, cơ chế lập lịch tài nguyên là một phần quan trọng để phân bổ tài nguyên trong hệ thống một cách hiệu quả và công bằng. YARN cung cấp ba cơ chế lập lịch chính cho việc phân phối tài nguyên giữa các ứng dụng (hoặc job) khác nhau trong cụm: FIFO Scheduler, Capacity Scheduler, và Fair Scheduler. Mỗi cơ chế có cách hoạt động và mục đích sử dụng khác nhau. Dưới đây là giải thích chi tiết về từng loại.

- FIFO Scheduler (First In, First Out)
 - Mô tả: FIFO là một cơ chế lập lịch đơn giản nhất, hoạt động theo nguyên lý "trước đến trước" (first-come, first-served). Các ứng dụng sẽ được cấp tài nguyên theo thứ tự mà chúng yêu cầu, mà không phân biệt mức độ ưu tiên của các ứng dụng hoặc tài nguyên có sẵn.
 - Cách hoạt động:
 - Các ứng dụng (hoặc job) sẽ được đưa vào hàng đợi.

HADOOP

- Các ứng dụng trong hàng đợi sẽ được cấp tài nguyên theo thứ tự lần lượt (đầu tiên đến trước được cấp tài nguyên trước).
- FIFO không phân biệt giữa các ứng dụng, và không có giới hạn về tài nguyên dành cho các nhóm người dùng hoặc các ứng dụng khác nhau.
 - Capacity Scheduler
 - Mô tả: Capacity Scheduler được thiết kế để cung cấp khả năng quản lý tài nguyên có thể mở rộng cho các tổ chức lớn. Nó phân bổ tài nguyên dựa trên các hàng đợi (queues), mỗi hàng đợi có thể được cấp một tỷ lệ tài nguyên cố định trong cụm.
 - Cách hoạt động:
 - Các tài nguyên trong hệ thống được chia thành các hàng đợi (queues), và mỗi hàng đợi sẽ có một tỷ lệ tài nguyên nhất định. Mỗi hàng đợi có thể đại diện cho một nhóm người dùng hoặc một loại ứng dụng.
 - Hàng đợi sẽ được phân bổ tài nguyên dựa trên các quy tắc tỷ lệ được định nghĩa trước. Ví dụ, nếu một hàng đợi được cấu hình để sử dụng 50% tài nguyên hệ thống, thì nó sẽ không bao giờ sử dụng quá 50% tài nguyên của hệ thống, bất kể có bao nhiêu job đang chờ trong hàng đợi đó.
 - Preemption (tạm ngừng) có thể được sử dụng để đảm bảo rằng các hàng đợi có tài nguyên cần thiết khi có các ứng dụng khác cần tài nguyên.
 - Fair Scheduler
 - Mô tả: Fair Scheduler nhằm đảm bảo tài nguyên được phân phối công bằng giữa tất cả các ứng dụng trong cụm, với mục tiêu tất cả các ứng dụng đều nhận được một lượng tài nguyên công bằng.
 - Cách hoạt động:
 - Fair Scheduler phân bổ tài nguyên một cách công bằng, tức là mỗi ứng dụng hoặc job sẽ nhận được một phần tài nguyên tương đối giống nhau.
 - Nếu có nhiều job đang chạy, Fair Scheduler sẽ phân bổ tài nguyên sao cho tất cả các job nhận được tài nguyên giống nhau (hoặc gần giống nhau), giúp tránh tình trạng "Starvation" cho các job nhỏ.
 - Dynamic Allocation: Nếu một ứng dụng hoặc job hoàn thành, tài nguyên mà nó chiếm dụng sẽ được phân bổ lại cho các ứng dụng khác trong cụm, đảm bảo rằng tài nguyên không bị lãng phí.
 - Công bằng theo các job: Fair Scheduler có thể được cấu hình để phân bổ tài nguyên theo từng job hoặc nhóm job, và đảm bảo rằng tài nguyên được chia đều.

4.3 Các cải tiến của Yarn trong Hadoop 3.x

Trong Hadoop 3.x, Apache YARN đã có một số cải tiến quan trọng nhằm nâng cao khả năng quản lý tài nguyên, cải thiện hiệu suất và khả năng mở rộng của hệ thống. Dưới đây là một số cải tiến nổi bật của YARN trong Hadoop 3.x:

- YARN Federation

HADOOP

Mô tả: YARN Federation cho phép mở rộng cụm Hadoop qua nhiều cụm YARN riêng biệt. Điều này giúp quản lý tài nguyên từ nhiều cụm YARN khác nhau và cung cấp khả năng mở rộng linh hoạt, giảm thiểu sự phụ thuộc vào một ResourceManager duy nhất.

Lợi ích:

Khả năng mở rộng lớn hơn: Cho phép mở rộng quy mô theo chiều ngang bằng cách kết hợp nhiều cụm YARN khác nhau.

Tăng tính sẵn sàng: Các cụm có thể hoạt động độc lập, giảm thiểu sự gián đoạn khi một cụm gặp sự cố.

Quản lý tài nguyên hiệu quả hơn: ResourceManager có thể phân bổ tài nguyên từ nhiều cụm, giúp tối ưu hóa sử dụng tài nguyên trong môi trường phân tán.

- NodeManager Enhancements
 - Mô tả: Trong Hadoop 3.x, NodeManager đã được cải thiện để hỗ trợ việc quản lý tài nguyên và hiệu suất tốt hơn.
 - Các cải tiến:
 - Quản lý tài nguyên CPU tốt hơn: NodeManager giờ đây có thể quản lý tài nguyên CPU theo mức độ chi tiết hơn, giúp phân bổ tài nguyên CPU chính xác hơn cho các ứng dụng.
 - Tài nguyên Docker và Virtualization: Hỗ trợ việc chạy các container trong các môi trường ảo hóa hoặc sử dụng Docker containers, giúp tận dụng các công nghệ mới trong việc quản lý và cô lập các ứng dụng.
 - Cải thiện việc giám sát: NodeManager có thể theo dõi và báo cáo về tình trạng tài nguyên sử dụng của các container chi tiết hơn, giúp phát hiện các vấn đề sớm.
 - GPU Scheduling
 - Mô tả: Hadoop 3.x đã thêm tính năng GPU Scheduling, cho phép YARN sử dụng và phân bổ tài nguyên GPU cho các ứng dụng yêu cầu xử lý tính toán song song (như deep learning).
 - Lợi ích:
 - Hỗ trợ AI và ML: Với việc hỗ trợ GPU, YARN có thể chạy các ứng dụng học sâu (deep learning) hoặc các ứng dụng yêu cầu tài nguyên tính toán mạnh mẽ như TensorFlow hoặc PyTorch.
 - Tối ưu tài nguyên: YARN có thể phân bổ và quản lý tài nguyên GPU cùng với CPU và bộ nhớ, tối ưu hóa hiệu suất của các ứng dụng tính toán song song.
 - Support for the Hadoop Distributed Data Model (HDFS) Snapshots
 - Mô tả: HDFS trong Hadoop 3.x hỗ trợ Snapshots, cho phép người dùng tạo các bản sao của dữ liệu hệ thống và khôi phục lại trong trường hợp gặp sự cố.
 - Lợi ích:
 - Khôi phục nhanh chóng: Các snapshot của dữ liệu giúp khôi phục dữ liệu nhanh chóng nếu có sự cố xảy ra trong khi chạy các ứng dụng.
 - Lưu trữ bảo mật: HDFS Snapshots có thể giúp bảo vệ dữ liệu, đặc biệt là trong

HADOOP

môi trường có yêu cầu độ bảo mật cao.

- Support for YARN ResourceManager High Availability
 - Mô tả: Hadoop 3.x cải thiện tính khả dụng cao cho ResourceManager (RM) thông qua việc hỗ trợ chế độ High Availability (HA). Điều này cho phép ResourceManager có thể hoạt động liên tục mà không có sự gián đoạn trong trường hợp một ResourceManager gặp sự cố.
 - Lợi ích:
 - Đảm bảo tính liên tục: Nếu ResourceManager chính bị lỗi, một ResourceManager phụ sẽ tự động trở thành chính và tiếp tục quản lý tài nguyên mà không làm gián đoạn hệ thống.
 - Tăng độ bền và độ tin cậy: HA giúp giảm thiểu điểm thất bại duy nhất và cải thiện khả năng chịu lỗi của hệ thống.
- YARN Timeline Service v2
 - Mô tả: Timeline Service v2 là một cải tiến lớn trong Hadoop 3.x, cung cấp dịch vụ ghi nhận, theo dõi và phân tích thông tin về các ứng dụng trong YARN.
 - Lợi ích:
 - Theo dõi hiệu suất tốt hơn: Cung cấp thông tin chi tiết hơn về tiến độ, tài nguyên sử dụng và các vấn đề trong quá trình thực thi của ứng dụng.
 - Giám sát linh hoạt: Hỗ trợ truy vấn và phân tích các dữ liệu lịch sử để tối ưu hóa hiệu suất và phát hiện vấn đề.
- Enhanced Fair Scheduler
 - Mô tả: Fair Scheduler trong Hadoop 3.x được cải tiến để cung cấp khả năng lập lịch tài nguyên tốt hơn, với các tính năng mới như việc phân chia tài nguyên chính xác hơn giữa các ứng dụng.
 - Lợi ích:
 - Công bằng hơn trong phân bổ tài nguyên: Với các cải tiến, Fair Scheduler có thể phân bổ tài nguyên một cách linh hoạt và công bằng hơn, đặc biệt trong môi trường có nhiều người dùng và ứng dụng.
 - Giảm thiểu tình trạng starvation: Cải thiện khả năng chia sẻ tài nguyên giữa các job, giúp giảm tình trạng một số ứng dụng chiếm dụng tài nguyên lâu dài, trong khi các ứng dụng khác bị bỏ rơi.
- YARN Resource Usage Metrics
 - Mô tả: Trong Hadoop 3.x, YARN cung cấp khả năng thu thập và báo cáo chi tiết về việc sử dụng tài nguyên của các ứng dụng.
 - Lợi ích:
 - Giám sát chi tiết: Cung cấp các chỉ số chi tiết về việc sử dụng tài nguyên (như bộ nhớ, CPU, và thời gian chờ), giúp người quản trị và người phát triển dễ dàng giám sát và tối ưu hóa tài nguyên.
 - Tối ưu hóa hiệu suất: Các chỉ số này giúp nhận diện các vấn đề hiệu suất, từ đó đưa ra các điều chỉnh cần thiết để tối ưu hóa việc phân bổ tài nguyên và giảm thiểu độ trễ.
- YARN Client Enhancements

HADOOP

- Mô tả: Trong Hadoop 3.x, có sự cải tiến trong việc triển khai client để tương tác với YARN, bao gồm việc dễ dàng triển khai và sử dụng ứng dụng YARN từ các hệ thống bên ngoài.

- Lợi ích:
 - Dễ dàng tích hợp với các hệ thống bên ngoài: Các cải tiến giúp YARN tương thích tốt hơn với các hệ thống bên ngoài và các công cụ quản lý tài nguyên khác.
 - Cải thiện tính linh hoạt: Các ứng dụng có thể sử dụng các giao diện và API cải tiến để tương tác hiệu quả hơn với YARN.

4.4 Ưu điểm và nhược điểm của YARN

4.4.1 Ưu điểm của YARN

- Quản lý tài nguyên linh hoạt và hiệu quả

YARN giúp quản lý tài nguyên (CPU, bộ nhớ, v.v.) giữa các ứng dụng trong cụm Hadoop một cách linh hoạt. Điều này giúp cải thiện hiệu quả sử dụng tài nguyên và tối ưu hóa việc phân bổ tài nguyên.

YARN hỗ trợ việc chạy các ứng dụng không phải MapReduce, ví dụ như Spark, Tez, và các ứng dụng khác. Điều này mở rộng khả năng sử dụng Hadoop để chạy nhiều loại ứng dụng khác nhau.

- Khả năng mở rộng và khả năng chịu lỗi cao

YARN có khả năng mở rộng quy mô theo chiều ngang rất tốt. Bạn có thể thêm nhiều NodeManager và ResourceManager để tăng cường khả năng xử lý và khả năng chịu lỗi.

Với ResourceManager High Availability và khả năng quản lý tài nguyên giữa nhiều cụm (YARN Federation), YARN có khả năng duy trì sự sẵn sàng và không bị gián

đoạn trong trường hợp có sự cố xảy ra.

- Hỗ trợ nhiều cơ chế lập lịch tài nguyên

YARN cung cấp nhiều cơ chế lập lịch tài nguyên khác nhau như FIFO, Capacity Scheduler, và Fair Scheduler, giúp người dùng có thể lựa chọn cơ chế phù hợp với yêu cầu và đặc điểm của ứng dụng hoặc tổ chức.

Điều này giúp cải thiện công bằng và tối ưu tài nguyên khi chạy nhiều ứng dụng trong một cụm Hadoop.

- Quản lý và theo dõi tài nguyên

YARN cung cấp các công cụ giám sát mạnh mẽ, bao gồm ResourceManager UI và NodeManager UI, giúp người quản trị dễ dàng theo dõi và kiểm soát việc sử dụng tài nguyên của các ứng dụng.

Timeline Service giúp ghi lại và theo dõi các sự kiện lịch sử trong hệ thống, hỗ trợ phân tích và tối ưu hóa hiệu suất.

- Hỗ trợ ứng dụng đa dạng

HADOOP

YARN không chỉ hỗ trợ các job MapReduce mà còn hỗ trợ các ứng dụng như Spark, Tez, Flink, và các framework phân tán khác. Điều này giúp YARN trở thành một nền tảng quản lý tài nguyên linh hoạt cho các ứng dụng phân tán khác nhau.

- Hỗ trợ GPU (với YARN 3.x)

YARN hỗ trợ GPU scheduling, giúp các ứng dụng yêu cầu tài nguyên tính toán mạnh mẽ như các ứng dụng học sâu (deep learning) có thể chạy trên các hệ thống với GPU, nâng cao hiệu suất tính toán.

4.4.2 Nhược điểm của YARN

- Cấu hình và quản lý phức tạp

YARN có thể khá phức tạp khi cấu hình và quản lý, đặc biệt khi hệ thống lớn hoặc khi yêu cầu cấu hình tùy chỉnh cho các loại ứng dụng khác nhau. Người quản trị cần phải hiểu rõ các thành phần của YARN và cách chúng tương tác với nhau.

Các tính năng như ResourceManager High Availability, YARN Federation có thể đòi hỏi kiến thức chuyên sâu và khả năng cấu hình chính xác để hoạt động ổn định.

- Tài nguyên không đồng đều trong môi trường đa ứng dụng

Trong môi trường có nhiều ứng dụng chạy đồng thời, việc phân bổ tài nguyên có thể trở nên không đồng đều nếu không có sự cấu hình hợp lý. Điều này có thể dẫn đến một số ứng dụng bị starved (không đủ tài nguyên để chạy) trong khi các ứng dụng khác chiếm hết tài nguyên.

Mặc dù Fair Scheduler giúp giải quyết vấn đề này, nhưng trong các môi trường phức tạp, việc đảm bảo phân bổ tài nguyên công bằng giữa các job vẫn là một thách thức.

- Khó khăn trong việc tối ưu hiệu suất

Trong các môi trường có nhiều ứng dụng và nhóm người dùng khác nhau, việc tối ưu hóa hiệu suất của YARN để đảm bảo tài nguyên được phân phối và sử dụng hiệu quả có thể gặp nhiều khó khăn. Các cấu hình không hợp lý có thể dẫn đến việc tài nguyên bị lãng phí hoặc sử dụng không hiệu quả.

Cần phải giám sát liên tục và điều chỉnh các thông số cấu hình để tối ưu hóa hiệu suất và sử dụng tài nguyên.

- Hiệu suất khi có quá nhiều job

Khi có một lượng lớn các job hoặc ứng dụng chạy đồng thời, YARN có thể gặp vấn đề về hiệu suất, đặc biệt là khi ResourceManager phải xử lý nhiều yêu cầu tài nguyên từ các ứng dụng khác nhau. Điều này có thể dẫn đến độ trễ cao trong việc phân bổ tài nguyên và làm giảm hiệu suất chung của hệ thống.

Cần phải tối ưu cấu hình và sử dụng các cơ chế lập lịch hợp lý để giảm thiểu các vấn đề này.

- Chưa hoàn toàn lý tưởng cho các workload real-time

HADOOP

Mặc dù YARN hỗ trợ nhiều loại ứng dụng, nhưng đối với các workload yêu cầu độ trễ thấp và real-time, như các ứng dụng streaming thời gian thực, YARN có thể không phải là giải pháp tối ưu. Các hệ thống như Apache Flink hoặc Apache Kafka Streams có thể được tối ưu hơn cho các workload này.

- Quản lý và lưu trữ dữ liệu phức tạp

Mặc dù YARN quản lý tài nguyên tốt, nhưng việc lưu trữ và truy xuất dữ liệu trong HDFS hoặc các hệ thống lưu trữ phân tán khác vẫn có thể gặp phải các vấn đề về độ trễ và hiệu suất, đặc biệt là khi dữ liệu lớn và phức tạp.

Phần 5 . Mô hình lập trình phân tán MapReduce

5.1 Giới thiệu về MapReduce

MapReduce là một khung làm việc xử lý dữ liệu phân tán được sử dụng để xử lý dữ liệu lớn trên **Hadoop**. **MapReduce** được phát triển bởi **Google** và sau đó được **Apache Software Foundation** phát triển và phát hành dưới dạng một phần của hệ sinh thái **Hadoop**.

5.2 Kiến trúc và cách thức hoạt động của MapReduce

MapReduce có 2 hàm chính là Map() và Reduce(), đây là 2 hàm đã được định nghĩa bởi người dùng và nó cũng chính là 2 giai đoạn liên tiếp trong quá trình xử lý dữ liệu của MapReduce. Nhiệm vụ cụ thể của từng hàm như sau:

- **Hàm Map():** có nhiệm vụ nhận Input cho các cặp giá trị/ khóa và output chính là tập những cặp giá trị/ khóa trung gian. Sau đó, chỉ cần ghi xuống đĩa cứng và tiến hành thông báo cho các hàm Reduce() để trực tiếp nhận dữ liệu.
- **Hàm Reduce():** có nhiệm vụ tiếp nhận từ khóa trung gian và những giá trị tương ứng với lượng từ khóa đó. Sau đó, tiến hành ghép chúng lại để có thể tạo thành một tập khóa khác nhau. Các cặp khóa/giá trị này thường sẽ thông qua một con trỏ vị trí để đưa vào các hàm reduce. Quá trình này sẽ giúp cho lập trình viên quản lý dễ dàng hơn một lượng danh sách cũng như phân bổ giá trị sao cho phù hợp nhất với bộ nhớ hệ thống.
- Ở giữa Map và Reduce thì còn 1 bước trung gian đó chính là **Shuffle**. Sau khi Map hoàn thành xong công việc của mình thì Shuffle sẽ làm nhiệm vụ chính là thu thập cũng như tổng hợp từ khóa/giá trị trung gian đã được map sinh ra trước đó rồi chuyển qua cho Reduce tiếp tục xử lý.

5.3 Ưu điểm và nhược điểm của MapReduce

5.3.1 Ưu điểm

- **Xử lý dữ liệu lớn:** MapReduce được thiết kế để xử lý và phân tích dữ liệu lớn bằng một cách hiệu quả, cho phép các công ty và tổ chức xử lý các dữ liệu có kích thước rất lớn mà trước đây chưa được thực hiện được.
- **Tính mở rộng:** MapReduce có khả năng mở rộng ngang, điều này có nghĩa là nó có thể chạy trên nhiều bài hát máy tính độc lập, giúp tăng tốc độ xử lý và xử lý dữ liệu đạt hiệu quả cao nhất.
- **Dung sai Tool:** Hệ thống Map Giảm khả năng chịu lỗi tốt, tức là nếu có một hoặc một số nút xử lý dữ liệu gặp sự cố, hệ thống vẫn có khả năng hoàn thành công việc một cách an toàn.
- **Phân tích dữ liệu:** Quá trình phân tích bài hát dữ liệu giữa các tác vụ bản đồ và sắp xếp theo khóa làm cho quá trình xử lý luôn trở nên hiệu quả và tối ưu.
- **Hỗ trợ nhiều loại dữ liệu:** MapReduce có khả năng xử lý một loạt các loại dữ liệu

5.3.2 Nhược điểm

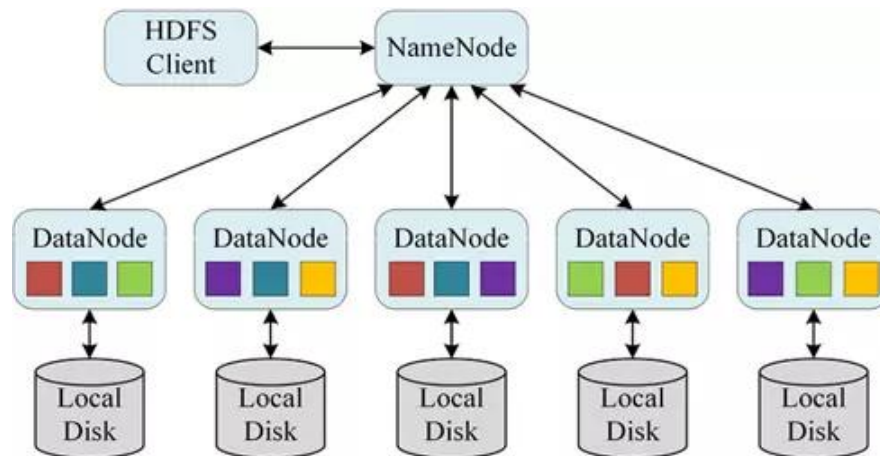
- **Hiệu suất chậm: I/O đĩa:** Một trong những nhược điểm lớn nhất của MapReduce là việc đọc và ghi dữ liệu từ đĩa giữa các bước Map và Reduce. Mỗi bước MapReduce yêu cầu việc truy xuất dữ liệu từ đĩa, điều này làm giảm hiệu suất khi so với các mô hình xử lý khác như in-memory processing (xử lý trong bộ nhớ), như Spark.
- **Shuffle và Sort:** Quá trình Shuffle và Sort giữa các bước Map và Reduce có thể gây tắc nghẽn hệ thống, đặc biệt khi lượng dữ liệu trung gian rất lớn.
- **Không thích hợp cho xử lý thời gian thực:** MapReduce được thiết kế cho batch processing (xử lý theo lô), nghĩa là các tác vụ được thực hiện theo từng đợt. Điều này không phù hợp với các ứng dụng yêu cầu xử lý thời gian thực hoặc gần thời gian thực, như phân tích dữ liệu stream hoặc xử lý sự kiện tức thời.
- **Khó khăn với các tác vụ phức tạp:** Mặc dù MapReduce rất mạnh mẽ trong các tác vụ đơn giản như đếm từ, tính tổng, nhưng khi đối mặt với các bài toán phức tạp hơn (ví dụ: tính toán đồ thị, phân tích dữ liệu máy học), MapReduce có thể trở nên khó khăn và kém hiệu quả.
- **Quản lý trạng thái phức tạp:** MapReduce không hỗ trợ trực tiếp việc duy trì trạng thái trong suốt quá trình xử lý (như việc lưu trữ trạng thái giữa các lần chạy của một task). Điều này có thể tạo ra khó khăn khi cần xử lý các tác vụ yêu cầu trạng thái liên tục.
- **Không phù hợp cho các phép toán có tính tương tác cao:** Các bài toán yêu cầu nhiều phép toán với các tác vụ phức tạp hoặc yêu cầu truy vấn dữ liệu nhiều lần sẽ gặp khó khăn khi sử dụng MapReduce. Quá trình MapReduce yêu cầu phải ghi và đọc dữ liệu từ đĩa sau mỗi bước, dẫn đến độ trễ cao cho các tác vụ tương tác.
- **Khó khăn trong việc tối ưu hóa:** Việc tối ưu hóa các tác vụ trong MapReduce có

HADOOP

thể khá khó khăn, đặc biệt khi làm việc với các tập dữ liệu lớn. Các thao tác như phân bổ công việc cho các nút trong cluster, tối ưu hóa dung lượng bộ nhớ, hay việc điều chỉnh các tham số như số lượng task, số lượng bản sao (replica), v.v., đều đòi hỏi kỹ năng và kinh nghiệm sâu.

Phần 6. HDFS – Hệ thống lưu trữ phân tán trong Hadoop

6.1 Giới thiệu về HDFS (Hadoop Distributed File System)



Hình 14. Cấu trúc của HDFS

HDFS (viết tắt Hadoop Distributed File System) là hệ thống lưu trữ dữ liệu chính được sử dụng bởi các ứng dụng trong hệ sinh thái Hadoop. Đây là một khung xử lý phân tán mã nguồn mở, được sinh ra để giải quyết bài toán quản lý các bể dữ liệu khổng lồ và hỗ trợ lưu trữ cho các ứng dụng phân tích dữ liệu chuyên sâu.

HDFS vận hành dựa trên mô hình Master-Slave. Trong đó, NameNode quản lý siêu dữ liệu, còn DataNode trực tiếp lưu trữ dữ liệu thật. Cấu trúc này giúp hệ thống tệp phân tán đạt hiệu suất truy cập dữ liệu cực cao.

HDFS cho phép mở rộng linh hoạt trên các cụm Hadoop lớn, đáp ứng nhu cầu tăng trưởng dữ liệu không giới hạn. Điểm đặc biệt của HDFS là được thiết kế để chạy ổn định trên các phần cứng thương mại giá rẻ, giúp doanh nghiệp tiết kiệm chi phí hạ tầng thay vì phải đầu tư vào các siêu máy tính đắt đỏ.

6.2 Cấu trúc và nguyên tắc hoạt động của HDFS

6.2.1 Cấu trúc của HDFS

Đúng như tên gọi "Distributed" (Phân tán), HDFS không phải là một cỗ máy đơn lẻ mà là một mạng lưới gồm nhiều máy tính kết nối với nhau, gọi là các Nodes. Hệ thống này được phân cấp rõ ràng như sau:

Namenode: Đóng vai trò như "tổng đài viên". Nó không chứa dữ liệu thật mà chỉ giữ bản đồ vị trí (Metadata). Nó biết chính xác file nào đã bị cắt thành các block nào và đang nằm ở Datanode nào. Mọi yêu cầu đọc/ghi của người dùng đều phải qua Namenode điều phối.

Datanode: Là "kho hàng" chứa dữ liệu thực tế. Dữ liệu tại đây được lưu dưới dạng các khối (Block) có kích thước cố định (thường là 128MB) đã được chia nhỏ và phân tán từ trước.

HDFS Cluster (Cụm máy chủ): Đây là tập hợp tổng thể của tất cả các Nodes. Cluster chịu trách nhiệm:

- Tiếp nhận file, tự động "băm" nhỏ thành các Block.
- Phân phối các Block này rải rác sang các Datanode.
- Cập nhật vị trí lưu trữ về cho Namenode quản lý.

Dựa trên "Hệ số nhân bản" (Replication Factor), Cluster đảm bảo một Block dữ liệu luôn có nhiều bản sao (Copies) nằm trên các Datanode khác nhau. Nếu một server gặp sự cố, hệ thống vẫn hoạt động bình thường nhờ các bản sao dự phòng này.

6.2.2 Nguyên tắc hoạt động của HDFS

Khác với mô hình truyền thống (mang dữ liệu về ứng dụng để xử lý), HDFS áp dụng triết lý ngược lại: Đưa tính toán đến nơi chứa dữ liệu. Điều này giúp tận dụng khả năng xử lý song song tại nhiều điểm khác nhau trong mạng lưới.

Quy trình lưu trữ thực tế (Ví dụ với 1 file nặng 360MB):

- **Bước 1:** Cắt nhỏ: Hệ thống sẽ chia file 360MB thành các khối (Block) tiêu chuẩn (mặc định là 128MB). Kết quả: Ta có 3 khối (128MB + 128MB + 104MB).
- **Bước 2:** Phân tán và nhân bản: Hệ thống lấy khối đầu tiên cùng các bản sao của nó (thường là 3 bản tổng cộng) và rải ngẫu nhiên lên các DataNode khác nhau dựa trên thuật toán cân bằng tải.
- **Bước 3:** Lặp lại quy trình: Quá trình trên tiếp tục với các khối còn lại cho đến khi toàn bộ file được lưu trữ xong.

Kết quả: NameNode sẽ lưu giữ "bản đồ" vị trí của tất cả các khối và bản sao. Nhờ đó, khi cần truy vấn, hệ thống có thể đọc và xử lý dữ liệu song song trên nhiều máy cùng lúc, mang lại tốc độ vượt trội.

6.3 Ưu điểm và nhược điểm của HDFS

6.3.1 Ưu điểm của HDFS

HDFS (Hadoop Distributed File System) có những ưu điểm nổi bật sau:

- **Phân tán dữ liệu:** HDFS cho phép dữ liệu được phân tán trên nhiều máy tính trong một cụm Hadoop. Điều này giúp tận dụng được tài nguyên lưu trữ của nhiều máy tính và cho phép lưu trữ các tệp dữ liệu có kích thước lớn.
- **Tính toán và phân tán song song:** HDFS cho phép tính toán và xử lý dữ liệu song song trên nhiều máy tính. Thay vì chỉ sử dụng một máy tính duy nhất để xử lý công việc, HDFS có thể chia nhỏ công việc và phân phối chúng cho nhiều máy tính trong cụm, giúp tăng hiệu suất và tốc độ xử lý.
- **Sao chép dữ liệu:** HDFS hỗ trợ sao chép dữ liệu trên nhiều nút để đảm bảo tính sẵn sàng và độ tin cậy cao. Khi một nút gặp sự cố, dữ liệu vẫn có thể được truy cập thông qua

bản sao dữ liệu trên các nút khác, giảm thiểu rủi ro mất dữ liệu.

- **Mở rộng theo chiều dọc:** HDFS cho phép mở rộng hệ thống bằng cách nâng cấp cấu hình của các máy tính trong cụm. Điều này gọi là mở rộng theo chiều dọc (scale up), và cho phép tăng khả năng xử lý và lưu trữ của hệ thống.

- **Mở rộng theo chiều ngang:** HDFS cũng hỗ trợ mở rộng hệ thống bằng cách thêm các máy tính mới vào cụm Hadoop, thay vì phải nâng cấp phần cứng của các máy tính hiện có. Điều này được gọi là mở rộng theo chiều ngang (scale out), giúp dễ dàng mở rộng hệ thống theo nhu cầu tăng trưởng dữ liệu.

Như vậy, HDFS là một hệ thống lưu trữ phân tán, có khả năng mở rộng, sao chép dữ liệu và xử lý dữ liệu song song. Điều này giúp nó trở thành một lựa chọn phổ biến cho việc lưu trữ và xử lý dữ liệu lớn trong các môi trường sử dụng Hadoop.

6.3.2 Nhược điểm của HDFS

1. Hiệu suất chậm với các tệp nhỏ:
 - HDFS được tối ưu hóa cho các tệp có kích thước lớn, nhưng nó không hiệu quả khi xử lý nhiều tệp nhỏ. Việc lưu trữ các tệp nhỏ có thể gây ra nhiều khối tệp nhỏ trong hệ thống, dẫn đến sự gia tăng trong quản lý metadata, làm giảm hiệu suất tổng thể.
 - HDFS không thích hợp cho các ứng dụng yêu cầu lưu trữ và xử lý nhiều tệp nhỏ (ví dụ, hệ thống tệp truyền thống của máy tính).
2. Không phù hợp với các ứng dụng yêu cầu truy cập dữ liệu ngẫu nhiên:
 - HDFS được thiết kế chủ yếu cho các tác vụ ghi một lần và đọc nhiều lần (write-once, read-many). Nếu một ứng dụng cần thực hiện nhiều thao tác ghi và sửa đổi dữ liệu trong tệp, HDFS không phải là lựa chọn tốt.
 - HDFS không hỗ trợ các thao tác như random writes (ghi ngẫu nhiên) một cách hiệu quả.
3. Khó khăn khi xử lý các tệp nhỏ và cập nhật dữ liệu:
 - Việc cập nhật một tệp trong HDFS rất khó khăn vì hệ thống không hỗ trợ ghi lại (append) dễ dàng như các hệ thống tệp truyền thống. Để thay đổi dữ liệu, người dùng phải tạo tệp mới và thay thế tệp cũ.
 - Việc thay đổi nội dung của một tệp không phải là thao tác dễ dàng trong HDFS, điều này có thể làm phức tạp các ứng dụng yêu cầu khả năng chỉnh sửa tệp nhanh chóng.
4. Không hỗ trợ tốt cho truy vấn thời gian thực:
 - HDFS chủ yếu được thiết kế cho các tác vụ batch processing (xử lý theo lô), vì vậy, nếu một ứng dụng cần truy vấn và xử lý dữ liệu theo thời gian thực hoặc gần thời gian thực, HDFS có thể không đáp ứng yêu cầu này.
 - Các hệ thống như Apache Kafka hoặc Apache Flink có thể là lựa chọn tốt hơn cho các ứng dụng yêu cầu xử lý dữ liệu theo thời gian thực.
5. Tăng độ phức tạp khi quản lý và bảo trì:
 - Mặc dù HDFS cung cấp khả năng chịu lỗi, nhưng việc duy trì một cluster HDFS

HADOOP

có thể trở nên phức tạp khi hệ thống ngày càng lớn hơn. Các vấn đề như phân phối tài nguyên, quản lý DataNode, đảm bảo sao lưu chính xác, và tối ưu hóa hiệu suất đều cần phải được xử lý cẩn thận.

- HDFS không có cơ chế tích hợp để tự động thay thế các máy chủ khi chúng gặp sự cố (mặc dù các công cụ bổ sung có thể giúp việc này).

6. Quản lý metadata:

- NameNode lưu trữ tất cả thông tin metadata của HDFS, bao gồm thông tin về các tệp, thư mục và các khối dữ liệu. Khi hệ thống phát triển và số lượng tệp và khối dữ liệu tăng lên, metadata cũng sẽ trở nên rất lớn, gây khó khăn cho việc duy trì và quản lý NameNode.

- Nếu NameNode gặp sự cố, có thể dẫn đến sự gián đoạn nghiêm trọng trong hoạt động của toàn bộ hệ thống.

7. Tăng chi phí lưu trữ với bản sao (Replication):

- Mặc dù cơ chế sao lưu giúp đảm bảo tính chịu lỗi, nhưng việc lưu trữ nhiều bản sao của mỗi khối dữ liệu làm tăng chi phí lưu trữ. Mặc định là 3 bản sao cho mỗi khối, điều này có thể là một vấn đề trong trường hợp dữ liệu rất lớn, nơi chi phí lưu trữ trở thành yếu tố quan trọng.

Phần 7. Một số công cụ và hệ sinh thái mở rộng của Hadoop

7.1 Apache Hive – Hệ thống kho dữ liệu trên Hadoop

Định nghĩa	Công cụ Data Warehouse dựa trên Hadoop, hỗ trợ truy vấn bằng HiveQL (SQL-like).
Mục đích	Giúp người dùng truy vấn dữ liệu lớn mà không cần viết MapReduce thủ công.
Kiến trúc	- HiveQL → Compiler → Execution Engine (MapReduce/Tez/Spark). - Metastore: Quản lý metadata.
Tính năng chính	- Hỗ trợ SQL-like queries (JOIN, GROUP BY...). - Làm việc với structured/semi-structured data. - Partitioning & Bucketing để tối ưu hóa.
Ưu điểm	- Dễ sử dụng cho người biết SQL. - Khả năng mở rộng tốt nhờ Hadoop. - Tích hợp nhiều định dạng (CSV, Parquet...).
Hạn chế	- Độ trễ cao (không phù hợp real-time). - Không hỗ trợ đầy đủ UPDATE/DELETE.
Ứng dụng	- Batch processing, Data Warehousing. - Phân tích log, ETL, Reporting.

7.2 Apache Pig – Ngôn ngữ lập trình dữ liệu lớn trên Hadoop

Định nghĩa	Nền tảng phân tích dữ liệu lớn sử dụng ngôn ngữ kịch bản Pig Latin để xử lý dữ liệu trên Hadoop.
Mục đích	Giảm độ phức tạp khi lập trình MapReduce bằng cách sử dụng cú pháp đơn giản, trực quan hơn.
Kiến trúc	- Pig Latin Script → Compiler → Execution Engine (MapReduce/Tez). - Hỗ trợ Grunt Shell (tương tác trực tiếp).
Tính năng chính	- Hỗ trợ ETL (Extract, Transform, Load). - Làm việc với structured & unstructured data. - Tối ưu hóa tự động (optimizer).
Ưu điểm	- Dễ học, viết ngắn gọn hơn MapReduce. - Linh hoạt với nhiều định dạng dữ liệu. - Tự động tối ưu truy vấn.
Hạn chế	- Hiệu suất thấp hơn Spark. - Không phù hợp cho truy vấn phức tạp như SQL (Hive tốt hơn).
Ứng dụng	- Tiền xử lý dữ liệu (data preprocessing). - Phân tích log, xử lý pipeline.

7.3 Apache HBase – Cơ sở dữ liệu NoSQL trên Hadoop

Trong thời đại bùng nổ dữ liệu số, các hệ quản trị cơ sở dữ liệu truyền thống tỏ ra kém hiệu quả trong việc xử lý khối lượng dữ liệu khổng lồ và đa dạng ngày nay. Apache HBase ra đời như một giải pháp NoSQL phân tán, được xây dựng trên nền tảng HDFS của Hadoop, giúp giải quyết các thách thức về lưu trữ và xử lý dữ liệu lớn.

Định nghĩa	Cơ sở dữ liệu NoSQL phân tán, có khả năng mở rộng ngang, chạy trên HDFS.
Kiến trúc	- Master-Slave - RegionServer quản lý vùng dữ liệu - Zookeeper điều phối
Mô hình dữ liệu	Dạng column-family Dữ liệu được tổ chức thành: Row key, Column Family, Column Qualifier, Timestamp
Tính năng chính	- Truy cập ngẫu nhiên real-time - Linear scalability - Tích hợp với MapReduce/Spark
Ưu điểm	- Hiệu suất cao với dữ liệu lớn - Khả năng mở rộng vô hạn - Hỗ trợ phiên bản hóa dữ liệu (versioning)
Hạn chế	- Không hỗ trợ ACID đầy đủ - Không có ngôn ngữ truy vấn phức tạp như SQL - Yêu cầu thiết kế row key cẩn thận
Ứng dụng	- Real-time analytics - Time-series data (log, sensor data) - Message systems

Phần 8. Thách thức và xu hướng phát triển của Hadoop và Big Data

8.1 Những thách thức khi triển khai Hadoop

Vấn đề bảo mật:

- Hadoop ban đầu không được thiết kế với bảo mật là ưu tiên hàng đầu dẫn đến các mối đe dọa an ninh và tấn công: tấn công mạo danh, từ chối dịch vụ (DoS), tập lệnh chéo trang web (XSS), ...

- Theo báo cáo, Hadoop dễ dàng bị tin tặc toàn cầu xác định thông qua việc dò tìm các phiên bản mở, với khoảng 5.307 cụm Hadoop đang tiếp xúc với mạng Internet khi cài đặt bảo mật bị tắt, khiến chúng trở thành mục tiêu dễ bị tấn công khai thác bởi tin tặc. Dưới đây là Bảng dữ liệu về các lỗ hổng khác nhau theo từng năm được phát hiện và báo cáo trong khuôn khổ Hadoop:

- Bảo vệ dữ liệu trong Hadoop là vấn đề cấp thiết. Để chống lại các cuộc tấn công và ngăn chặn trộm cắp dữ liệu, các dự án bảo mật khác nhau như Apache Sentry, Apache Ranger, và Apache Knox và Rhino được đưa vào Hadoop ở giai đoạn cấp độ bảo mật cấp 3.

Kerberos cũng được tích hợp với AD và LDAP.

- Hiệu suất:
- Xử lý dữ liệu lớn đòi hỏi tài nguyên phần cứng mạnh và cấu hình tối ưu.
- Việc phân tán dữ liệu và tính toán có thể gây ra độ trễ nếu không được quản lý tốt.
- Có thể nâng cao hiệu suất Hadoop thông qua việc sắp xếp dữ liệu trong cụm

Hadoop không đồng nhất, sử dụng MapReduce cho dữ liệu lớn,...

- Độ phức tạp:
- Cài đặt, cấu hình, và quản lý Hadoop đòi hỏi kiến thức chuyên sâu.
- Việc tích hợp Hadoop với các hệ thống hiện có (như cơ sở dữ liệu truyền thống) cũng là một thách thức.

8.2 Các phương pháp tối ưu hóa Hadoop

- Tối ưu hóa phần cứng:
- Sử dụng ổ cứng SSD thay vì HDD để tăng tốc độ đọc/ghi.
- Tăng băng thông mạng để giảm độ trễ trong cụm.
- Theo báo cáo của Intel, SSD có thể tăng tốc độ đọc/ghi dữ liệu lên đến 5 lần so với HDD.

- Điều chỉnh các tham số trong tệp cấu hình như hdfs-site.xml, yarn-site.xml.
- Sử dụng các công cụ giám sát như Apache Ambari để theo dõi hiệu suất.
- Sử dụng công cụ hỗ trợ:
- Apache Spark để xử lý dữ liệu trong bộ nhớ (in-memory processing).
- Apache Tez để tối ưu hóa các tác vụ MapReduce.

HADOOP

- Theo Databricks, Spark có thể xử lý dữ liệu nhanh hơn Hadoop MapReduce gấp 100 lần trong một số tác vụ.
- Phân vùng dữ liệu: Chia nhỏ dữ liệu thành các phần nhỏ hơn để xử lý song song hiệu quả.

8.3 Hadoop có đang dần bị thay thế bởi các công nghệ mới không?

- Cạnh tranh từ các công nghệ mới:
 - Apache Spark: Xử lý dữ liệu nhanh hơn nhờ tính toán trong bộ nhớ.
 - Cloud Data Warehouses: Amazon Redshift, Google BigQuery, Snowflake.
 - Stream Processing: Apache Kafka, Apache Flink.
 - Cloud-native services: AWS EMR, Azure HDInsight giúp triển khai Hadoop dễ dàng hơn, nhưng các dịch vụ như Snowflake, BigQuery cạnh tranh trực tiếp.
- Kubernetes: Spark, Flink chạy trên Kubernetes thay vì YARN, giảm phụ thuộc vào Hadoop.
- Xu hướng real-time: Kafka, Flink thay thế MapReduce cho xử lý stream.

8.4 Những cải tiến trong các phiên bản mới của Hadoop

- Cải thiện hiệu suất: Các phiên bản mới của Hadoop tập trung vào cải thiện hiệu suất thông qua tối ưu hóa HDFS, YARN, và các thành phần khác.
- Bảo mật nâng cao: Tích hợp các tính năng bảo mật mạnh mẽ hơn như mã hóa dữ liệu end-to-end, quản lý quyền truy cập chi tiết, và hỗ trợ Kerberos.
- Hỗ trợ container hóa: Hadoop đang dần hỗ trợ tốt hơn cho việc chạy trên các nền tảng container như Docker và Kubernetes.
- Tích hợp với AI/ML: Các phiên bản mới của Hadoop đang tích hợp sâu hơn với các công cụ AI và machine learning, giúp xử lý dữ liệu thông minh hơn.
- Quản lý tài nguyên hiệu quả hơn: YARN được cải tiến để quản lý tài nguyên hiệu quả hơn, hỗ trợ các tải công việc phức tạp và đa dạng. Xu hướng thay thế hoặc mở rộng Hadoop (Cloud Computing, Kubernetes, AI Driven Data Processing,...)
- Điện toán đám mây (Cloud Computing): Các dịch vụ đám mây như AWS, Google Cloud, và Azure đang cung cấp các giải pháp thay thế linh hoạt và dễ quản lý hơn so với Hadoop truyền thống.
- Kubernetes: Kubernetes đang trở thành nền tảng phổ biến để quản lý các ứng dụng phân tán, bao gồm cả các ứng dụng xử lý dữ liệu lớn. Nó có thể thay thế hoặc bổ sung cho YARN trong việc quản lý tài nguyên.
- AI-Driven Data Processing: Các công nghệ xử lý dữ liệu dựa trên AI và machine learning đang dần thay thế các phương pháp xử lý dữ liệu truyền thống, giúp tối ưu hóa và tự động hóa quá trình phân tích dữ liệu.
- Serverless Computing: Serverless computing đang trở thành xu hướng, cho phép xử lý dữ liệu mà không cần quản lý cơ sở hạ tầng, giảm thiểu chi phí và độ phức tạp.

- Data Lakes và Data Warehouses hiện đại: Các giải pháp Data Lake hiện đại như Delta Lake, Apache Iceberg, và Data Warehouses như Snowflake đang dần thay thế HDFS và các công cụ truy vấn truyền thống của Hadoop.

8.5 Vai trò của Hadoop trong chiến lược dữ liệu của doanh nghiệp hiện đại

- Lưu trữ dữ liệu lớn: Trong môi trường kinh doanh dựa trên dữ liệu ngày nay, ngày càng có nhiều người chuyển sang Hadoop để quản lý dữ liệu lớn. Hadoop cung cấp khả năng lưu trữ và xử lý phân tán mạnh mẽ thông qua Hệ thống tệp phân tán Hadoop (HDFS) và mô hình lập trình MapReduce. Sự kết hợp này cho phép các doanh nghiệp xử lý khối lượng dữ liệu khổng lồ một cách hiệu quả, biến Hadoop thành một công cụ thiết yếu cho quản lý dữ liệu hiện đại.
- Xử lý dữ liệu phân tán: Hadoop cung cấp nền tảng để xử lý dữ liệu phân tán với chi phí thấp, phù hợp với các doanh nghiệp có nhu cầu xử lý lượng dữ liệu khổng lồ.
- Tích hợp với các công nghệ mới: Hadoop được tích hợp với các công nghệ mới như AI, machine learning, và điện toán đám mây để tạo ra các giải pháp dữ liệu toàn diện. Ví dụ các thuật toán AI có thể được sử dụng kết hợp với Hadoop để phân tích các tập dữ liệu lớn và khám phá các mẫu khó phát hiện theo cách thủ công. Sự kết hợp này giữa Hadoop và AI có thể dẫn đến phân tích dự đoán chính xác hơn, cải thiện thông tin chi tiết về khách hàng và tối ưu hóa hoạt động kinh doanh.
- Chiến lược dữ liệu đa dạng: Trong chiến lược dữ liệu của doanh nghiệp hiện đại, Hadoop thường được sử dụng như một phần của kiến trúc dữ liệu đa dạng, kết hợp với các công nghệ khác như Data Lakes, Data Warehouses, và các công cụ phân tích thời gian thực.
- Tiết kiệm chi phí: Hadoop vẫn là một lựa chọn tiết kiệm chi phí cho các doanh nghiệp cần xử lý và lưu trữ dữ liệu lớn mà không muốn đầu tư vào các giải pháp đắt tiền khác.

Phần 9. Kết luận

9.1 Tổng quan về Hadoop, YARN, MapReduce và Hệ sinh thái Big Data

- Hadoop: Là nền tảng mã nguồn mở xử lý phân tán dành cho dữ liệu có khối lượng lớn (PB-scale), đa dạng về cấu trúc (unstructured/semi-structured). Hadoop dựa trên HDFS (Hadoop Distributed File System) để lưu trữ dữ liệu phân tán trên nhiều node, kết hợp khả năng mở rộng ngang (horizontal scaling) và tính chịu lỗi (fault tolerance) cao.
- YARN (Yet Another Resource Negotiator): Đóng vai trò là cơ chế quản lý tài nguyên trong Hadoop 2.x+, tách biệt vai trò quản lý tài nguyên và xử lý dữ liệu. YARN cho phép tích hợp đa dạng engine xử lý (MapReduce, Spark, Flink) trên cùng một cluster, tối ưu hiệu năng và linh hoạt trong việc triển khai các workload khác nhau.
- MapReduce: Mô hình lập trình xử lý theo lô (batch processing) dựa trên hai pha Map và Reduce, phù hợp cho các tác vụ xử lý dữ liệu quy mô lớn. Dù tốc độ không bằng các công nghệ in-memory như Spark, MapReduce vẫn là giải pháp hiệu quả về chi phí cho bài toán xử lý phân tán truyền thống.
- Hệ sinh thái Big Data:
 - Hive & Pig: Cung cấp lớp abstraction để truy vấn dữ liệu bằng SQL-like (HiveQL) hoặc script (Pig Latin), tối ưu cho người dùng không chuyên về lập trình.
 - HBase: Cơ sở dữ liệu NoSQL hướng cột, hỗ trợ truy xuất real-time với độ trễ thấp.
 - Spark & Flink: Framework xử lý in-memory và streaming, khắc phục hạn chế về tốc độ của MapReduce.
 - Kafka & ZooKeeper: Quản lý luồng dữ liệu thời gian thực (Kafka) và phối hợp phân tán (ZooKeeper).

9.2 Trường hợp ứng dụng Hadoop

Hadoop là giải pháp tối ưu cho các kịch bản sau:

Xử lý dữ liệu phi cấu trúc/quy mô lớn: Phân tích log hệ thống, dữ liệu cảm biến IoT, hoặc dữ liệu đa phương tiện (hình ảnh, video).

Batch processing: ETL (Extract-Transform-Load), xử lý dữ liệu lịch sử, hoặc phân tích tổng hợp theo lô (ví dụ: báo cáo doanh thu hàng tháng).

Yêu cầu scalability: Dễ dàng mở rộng bằng cách thêm node phần cứng phổ thông, phù hợp với doanh nghiệp có ngân sách hạn chế.

Lưu trữ dài hạn: HDFS cung cấp khả năng lưu trữ tin cậy, chi phí thấp hơn so với hệ thống SAN/NAS.

Xử lý phân tán địa lý: Dữ liệu được phân tán trên nhiều data center, giảm latency khi truy

cập cục bộ.

9.3 Xu hướng phát triển của Hadoop và Big Data trong bối cảnh AI và Điện toán đám mây (Cloud)

Hội tụ Big Data và AI/ML:

Data Lake trên nền HDFS: Hadoop trở thành nền tảng lưu trữ tập trung cho dữ liệu thô, hỗ trợ xây dựng pipeline ML (Machine Learning) từ tiền xử lý đến huấn luyện mô hình.

Tích hợp framework AI: Spark MLlib, TensorFlow on YARN, hay Horovod tận dụng tài nguyên Hadoop để huấn luyện mô hình phân tán.

Kiến trúc Cloud-Native

Managed Hadoop dịch vụ: AWS EMR, Azure HDInsight, và Google Dataproc giúp triển khai cluster Hadoop trên cloud chỉ với vài click, tự động scaling theo nhu cầu.

Hybrid Cloud: Kết hợp lưu trữ on-premise (HDFS) với xử lý đám mây để cân bằng giữa bảo mật và hiệu suất.

Cạnh tranh và tiến hóa:

Spark/Flink thay thế MapReduce trong xử lý streaming/real-time, nhưng Hadoop vẫn giữ vai trò lưu trữ.

Container hóa với Kubernetes: Hadoop tận dụng Kubernetes để quản lý container, tăng tính linh hoạt và tối ưu hóa tài nguyên.

Tối ưu hóa phần cứng:

GPU/TPU Acceleration: Tích hợp GPU vào cluster Hadoop để tăng tốc xử lý các workload AI/Deep Learning.

Edge Computing: Xử lý dữ liệu tại biên (edge nodes) kết hợp với HDFS để giảm tải bandwidth.

➤ **Tổng kết:** Hadoop vẫn là trụ cột trong hệ sinh thái Big Data nhờ kiến trúc phân tán mạnh mẽ và khả năng mở rộng vượt trội, nhưng để duy trì vị thế này, nó buộc phải tích hợp sẵn các lớp bảo mật toàn diện – từ bảo vệ dữ liệu nhạy cảm, đào tạo nhân sự, đến thiết kế hệ thống phản hồi bảo mật trực quan – thay vì yêu cầu cấu hình rời rạc. Trong bối cảnh công nghệ mới như Spark hay Serverless đang định hình lại cách xử lý dữ liệu, Hadoop cần tối ưu hóa khả năng tích hợp đa nền tảng (AI/Cloud) và kết hợp linh hoạt với Data Lake, Cloud-Native, cùng AI-Ready Infrastructure. Chính sự hòa quyện giữa nền tảng kỹ thuật vững chắc, cơ chế bảo mật "ẩn" nhưng mạnh mẽ, và xu hướng công nghệ tương lai sẽ biến Hadoop thành xương sống linh hoạt cho kỷ nguyên số hóa đa tầng – nơi quy mô dữ liệu và mức độ bảo mật phải tiến hóa song hành.

Tài liệu tham khảo:

1. Big data là gì? Đặc điểm 7V và ứng dụng tuyệt vời của Dữ liệu lớn
2. [Big Data là gì? Đặc trưng và ứng dụng của dữ liệu lớn](#)
3. <https://hadoop.apache.org/>
4. <https://hadoop.apache.org/docs/stable/hadoop-projectdist/hadoopcommon/SingleCluster.html>
5. <https://hadoop.apache.org/docs/stable/hadoop-projectdist/hadoopcommon/ClusterSetup.html>
6. <https://viblo.asia/p/huong-dan-cai-dat-hadoop-tren-ubuntu-1VgZvQ1vIAw>
7. <https://tek4.vn/huong-dan-cai-dat-hadoop-single-node/>
8. <https://viblo.asia/p/hdfs-la-gi-cach-hoat-dong-cua-hdfs-6J3Zg0QJIKm>
9. <https://tek4.vn/huong-dan-su-dung-hdfs-co-ban/>
10. <https://topdev.vn/blog/mapreduce-la-gi/>
11. <https://viblo.asia/p/viet-chuong-trinh-mapreduce-don-gian-6J3Zg0QJIKm>
12. <https://topdev.vn/blog/so-sanh-hadoop-va-spark/>
13. <https://www.sciencedirect.com/science/article/pii/S2590005619300025>
14. <https://journalofbigdata.springeropen.com/articles/8.1186/s40537-020-00388-5>
15. <https://theswissquality.ch/harnessing-hadoop-for-big-data-management-in-modern-enterprises/>
16. <https://ieeexplore.ieee.org/document/10837232>
17. https://www.researchgate.net/profile/Zahid-Ansari-2/publication/321537152_Apache_Pig_-_A_Data_Flow_Framework_Based_on_Hadoop_Map_Reduce/links/5afd45cba6fdcc3a5a275c41/Apache-Pig-A-Data-Flow-Framework-Based-on-Hadoop-Map-Reduce.pdf
18. <https://ieeexplore.ieee.org/abstract/document/6182030>
19. https://scholar.google.com/scholar?as_q=An+overview+of+the+Hadoop+%2F+MapReduce+%2F+HBase+framework+and+its+current+applications+in+bioinformatics&as_occt=title&hl=en&as_sdt=0%2C31
20. NOSQL Databases
21. [What is NoSQL? | Definition from TechTarget](#)
22. [NOSQL là gì? Một số ưu điểm và nhược điểm cần biết về nosql](#)
23. Cơ sở dữ liệu NoSQL là gì? – Giải thích cơ sở dữ liệu phi quan hệ – AWS
24. [NoSQL – Wikipedia tiếng Việt](#)
25. [No! to SQL and No! to NoSQL | So Many Oracle Manuals, So Little Time](#)
26. History repeats itself: sensible and NonsenSQL aspects of the NoSQL hoopla
27. [Các loại cơ sở dữ liệu NoSQL: Định nghĩa và Ứng dụng - ITviec Blog](#)

HADOOP

28. [SQL và NoSQL: Sự khác nhau và tình huống sử dụng](#)
29. [What Is NoSQL? NoSQL Databases Explained | MongoDB](#)
30. [myNoSQL • NoSQL Databases and Polyglot Persistence: A Curated Guide](#)
31. [Hadoop là gì? Hiểu thêm về kiến trúc của Hadoop | TopDev](#)
32. [Hadoop YARN: Architecture | Facebook](#)
33. [Tìm Hiểu Về Hadoop, HDFS, Hadoop MapReduce - Viblo](#)
34. [Mapreduce là gì? Tổng quan mô hình lập trình Mapreduce](#)
35. [HDFS là gì? Khái niệm, kiến trúc và ứng dụng thực tế trong Big data](#)